t = 0.00 | pure noise

From Noise to Signal

CS180 Project 3, Flow Matching

AUTHOR
Allena Oglivie

COURSE
CS180, Computer Vision

Flow models turn pure Gaussian noise into images by following a learned velocity field from t = 0 (noise) to t = 1 (a clean image). In Part A I use a pretrained text-conditioned flow model to denoise, sample, guide, and edit images. In Part B I build and train UNet flow models from scratch on MNIST, first as a one-step denoiser and then as time- and class-conditioned flow matching models. Like the results, this page starts in noise and resolves as you read.

CONTENTS

Part A: The Power of Flow Models

A.1.1 Constructing Noisy Training Inputs

A.1.2 Classical Denoising

A.1.3 One-Step Denoising

A.1.4 Visualizing Velocity

A.1.5	Iterative Denoising with Euler Sampling
A.1.6	Generating Images from Noise
A.2.1	Classifier-Free Guidance
A.3.1	Image-to-Image Editing
A.3.2	Text-Guided Image Editing
A.3.3	Editing Hand-Drawn and Web Images
A.3.4	Visual Anagrams
A.3.5	Hybrid Images

Part B: Flow Matching from Scratch

B.1.1	UNet
B.1.2	Noising Process
B.1.2.1	Training the Denoiser
B.1.2.2	Out-of-Distribution Testing
B.1.2.3	Denoising Pure Noise
B.2.3	Time-Conditioned Flow Matching
B.2.6	Class-Conditioned Flow Matching
t = 1.00	Conclusion

Part A

The Power of Flow Models

`t = 0.10 to 0.90`

`A.1.1`

Constructing Noisy Training Inputs

`t = 0.10`

My image is a Silent Princess flower screenshot from Breath of the Wild (1000x563, center-cropped to 512x512). It is mostly saturated green with a soft blurry background, and the flower has strong color and edge contrast. Noisy inputs interpolate between noise and the image:

```
x_t = (1 - t) * eps + t * x1 (t = 0 is noise, t = 1 is the
clean image)
```

Endpoint checks with `torch.allclose` pass: t = 0 returns the noise and t = 1 returns the image.

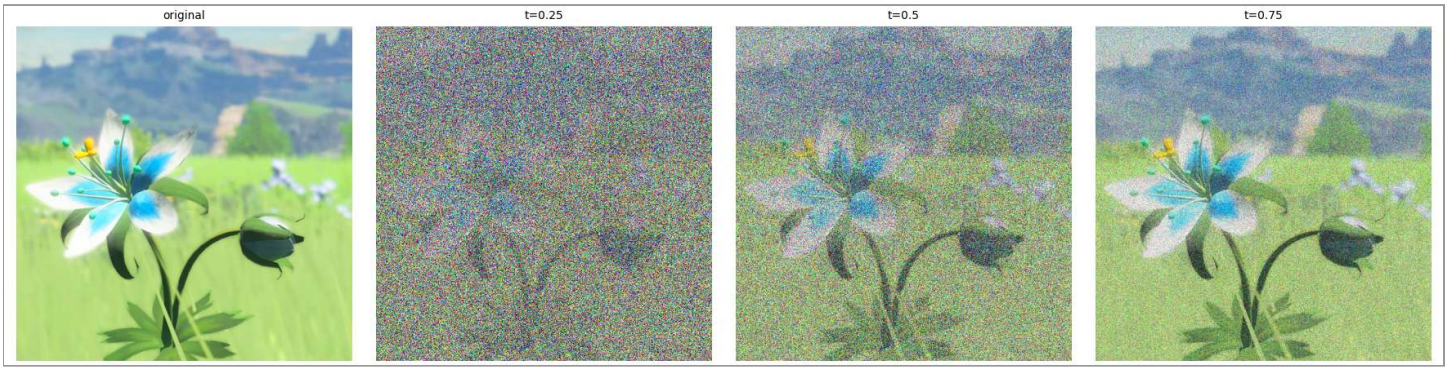


Fig A.1.1 | Silent Princess | original, then x_t at t = 0.25, 0.50, 0.75

- **t = 0.25:** almost all noise, only a faint outline of the stems.
- **t = 0.50:** flower, bud, and background shapes clearly visible but grainy.
- **t = 0.75:** mostly the image with a light noise layer.

Structure is already recognizable by t = 0.5.

A.1.2

Classical Denoising

`t = 0.17`

Dividing by t restores the image amplitude, and a Gaussian blur then averages out the remaining noise. I picked the best sigma by eye for each t.

```
x_t / t = x1 + ((1 - t) / t) * eps
```

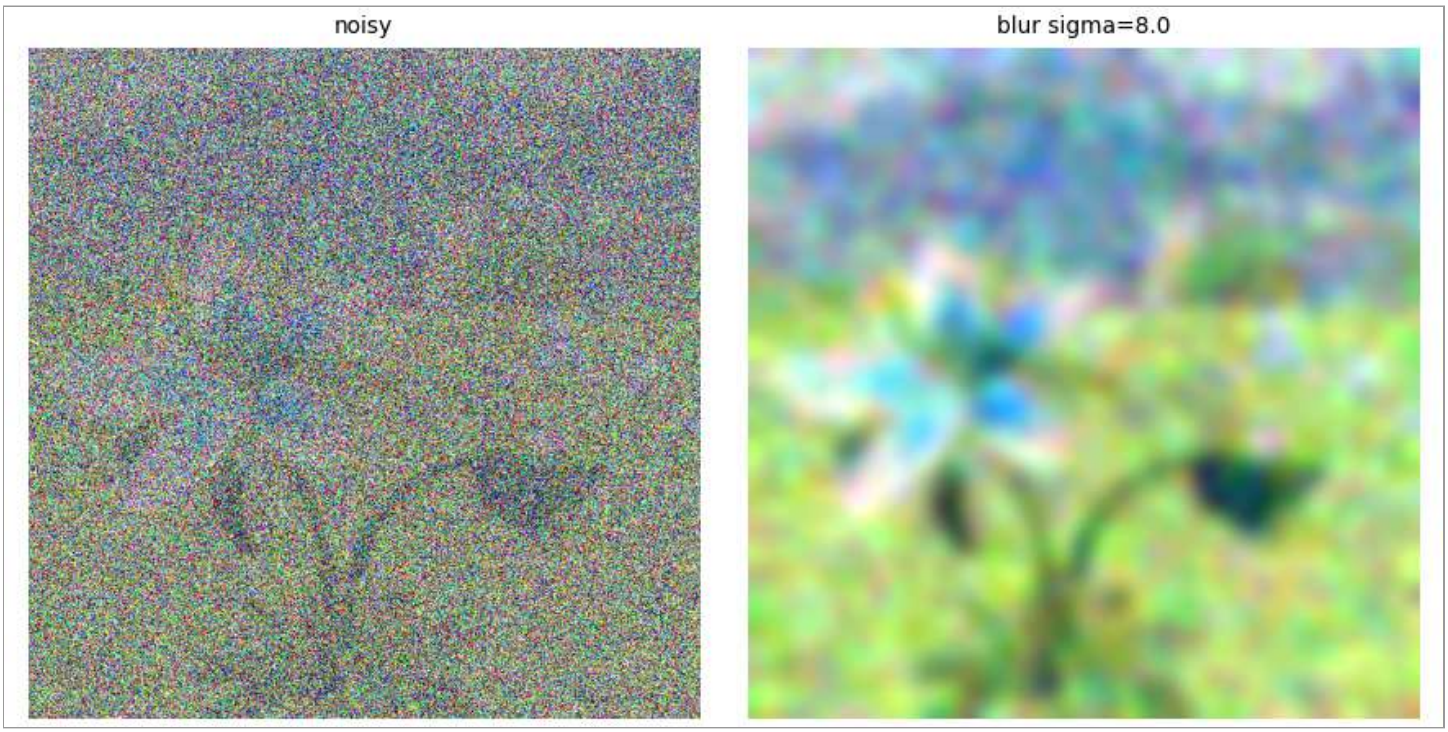


Fig A.1.2a | t = 0.25 | noisy input and Gaussian blur, sigma = 8.0



Fig A.1.2b | t = 0.50 | noisy input and Gaussian blur, sigma = 4.0

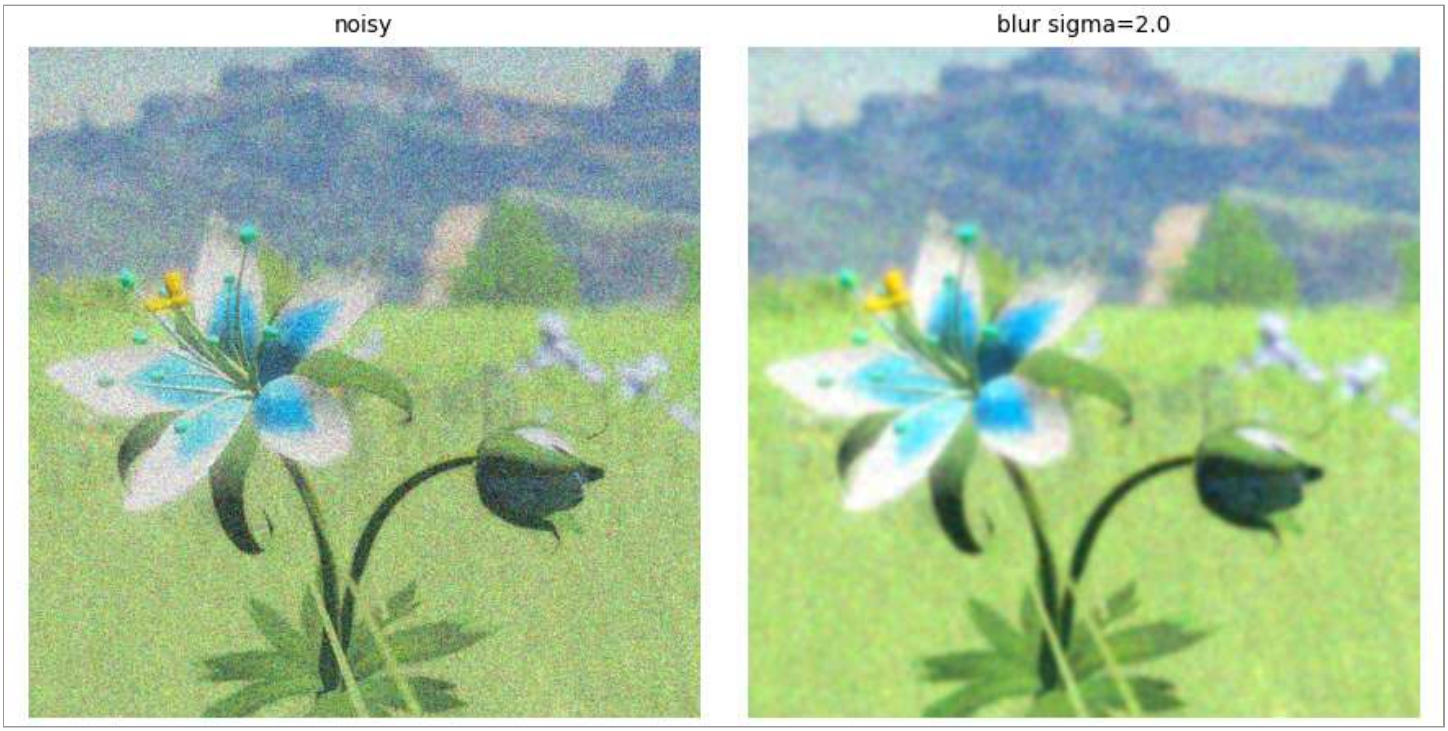


Fig A.1.2c | t = 0.75 | noisy input and Gaussian blur, sigma = 2.0

The noise factor after dividing is (1 - t) / t: 3 at t = 0.25 (noise three times stronger than the image), 1 at t = 0.5 (equal strength), and 1/3 at t = 0.75 (noise is weak). As predicted, the closer t is to 0, the larger sigma needs to be to average out the noise.

The tradeoff is that a larger sigma removes more noise but also smears edges and detail. Classical blur has no idea what a flower should look like, so it can only trade noise for blur.

A.1.3

One-Step Denoising

`t = 0.24`

The pretrained model predicts a velocity with the prompt `"a high quality photo"`, and one step jumps straight to t = 1. With the true velocity v = x1 - eps, this gives back x1 exactly. It uses exactly one model prediction per noisy input.

```
x1_hat = x_t + (1 - t) * v_theta(x_t, t, c)
```

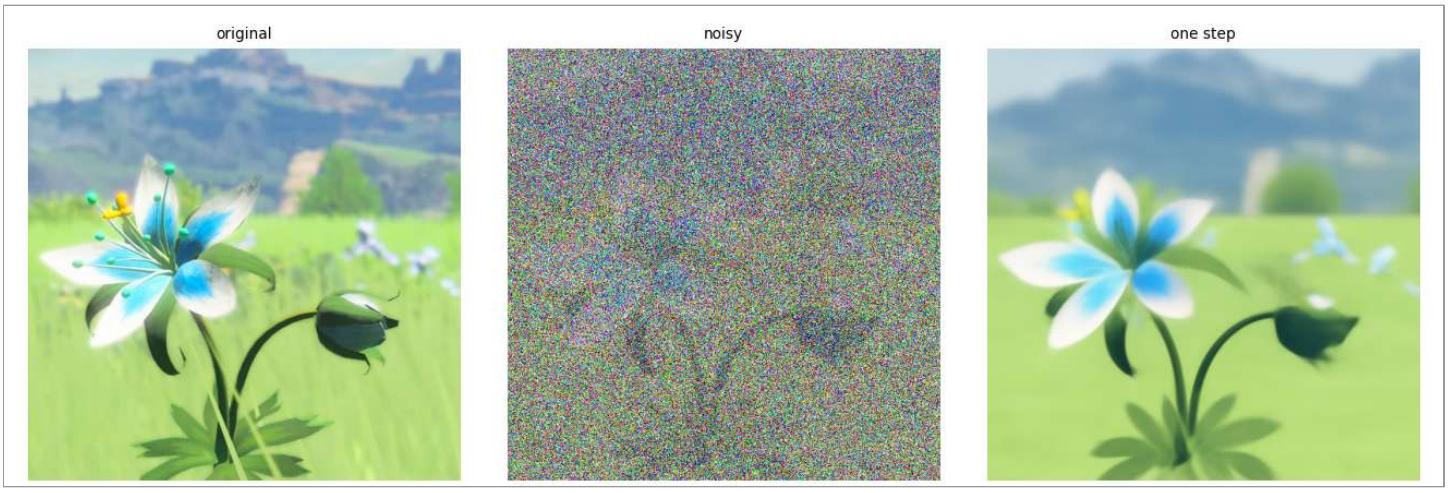


Fig A.1.3a | t = 0.25 | original, noisy, one-step estimate

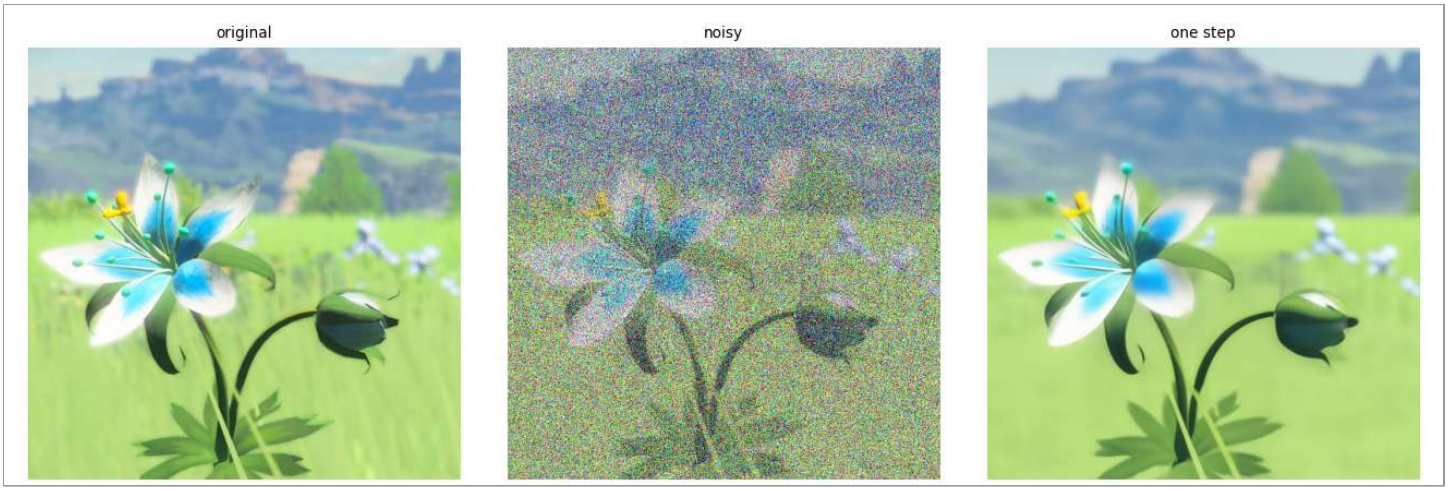


Fig A.1.3b | t = 0.50 | original, noisy, one-step estimate

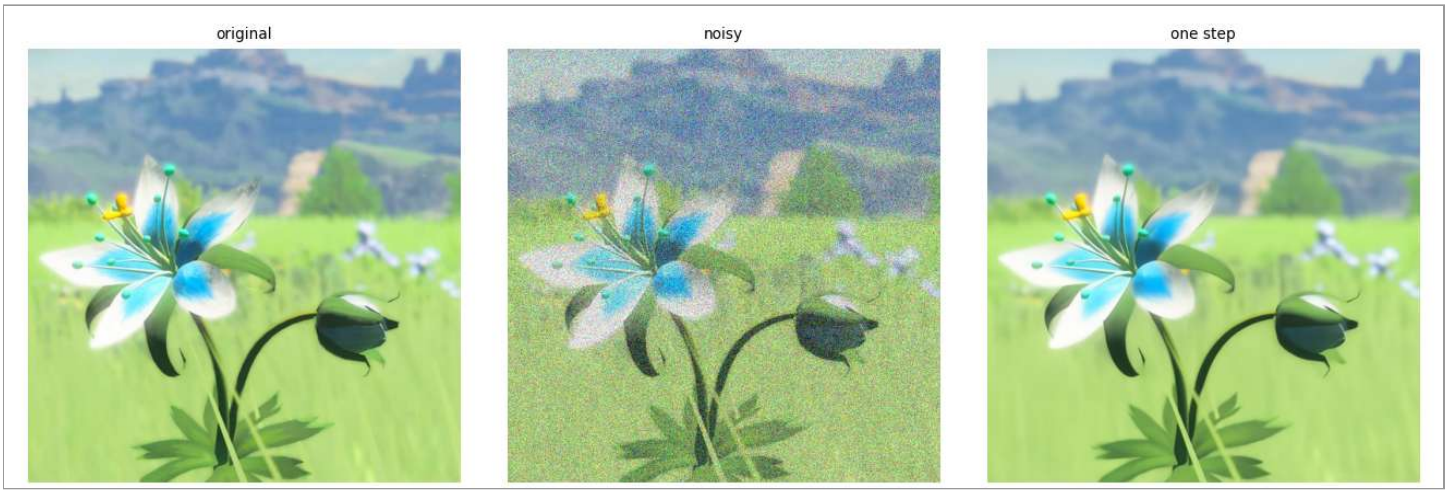


Fig A.1.3c | t = 0.75 | original, noisy, one-step estimate

At t = 0.25 the overall layout (flower, bud, stem, leaves, mountains, sky) and colors (blue petal centers, green field) are kept. The teal stamen tips, most of the orange center, and the petal and bud texture are lost, and the background flowers became blue smudges. The result is smooth and slightly blurry, but far better than Gaussian blur at the same noise level.

With high noise the model gives an averaged result: it is an average of all clean images that could plausibly produce this noisy input. Shared features survive and differing details cancel out. With lower noise, finer detail remains.

A.1.4

Visualizing Velocity

```
t = 0.32
```

I compare the predicted update (1 - t) * v_hat with the ground-truth update (1 - t) * v_gt, where v_gt = x1 - eps. The error map is the per-pixel RMS over RGB, shown with the same color limits in every row.

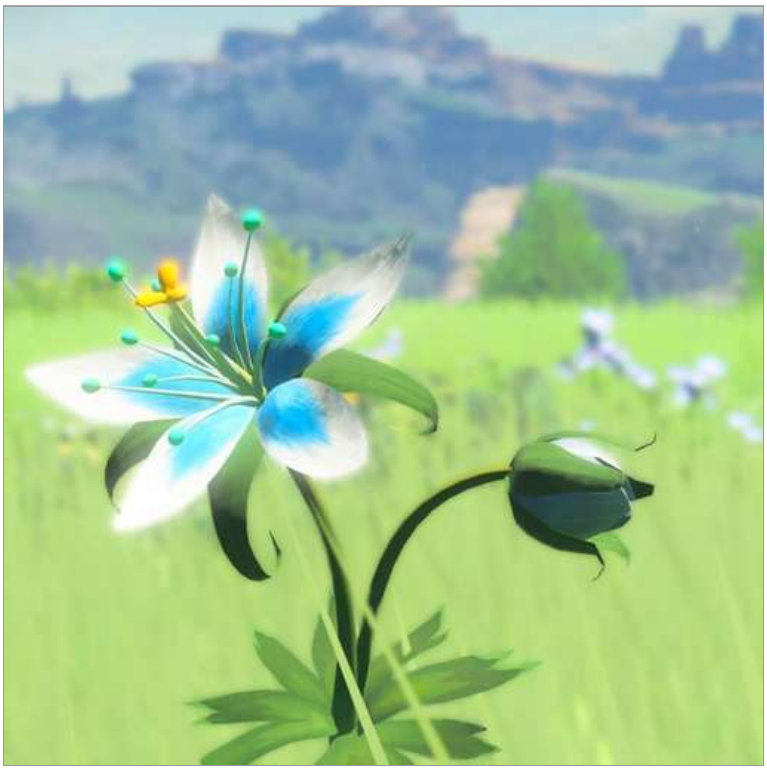


Fig A.1.4a | original image

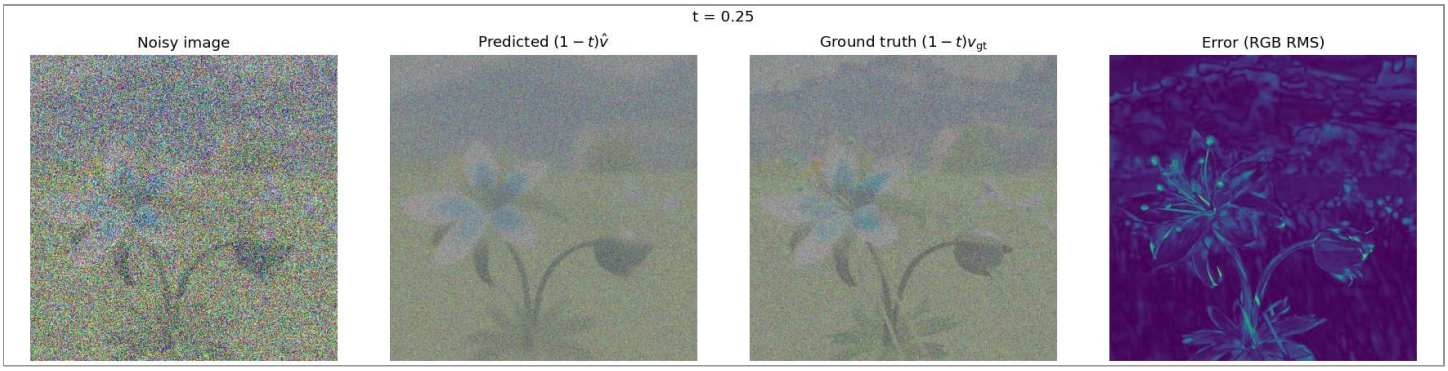


Fig A.1.4b | t = 0.25 | noisy, predicted update, ground-truth update, error

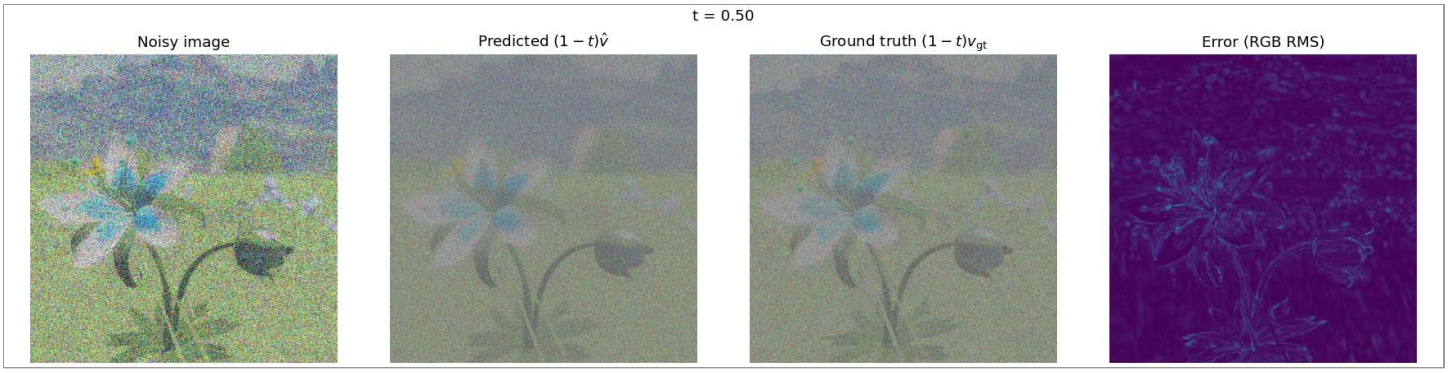


Fig A.1.4c | t = 0.50 | noisy, predicted update, ground-truth update, error

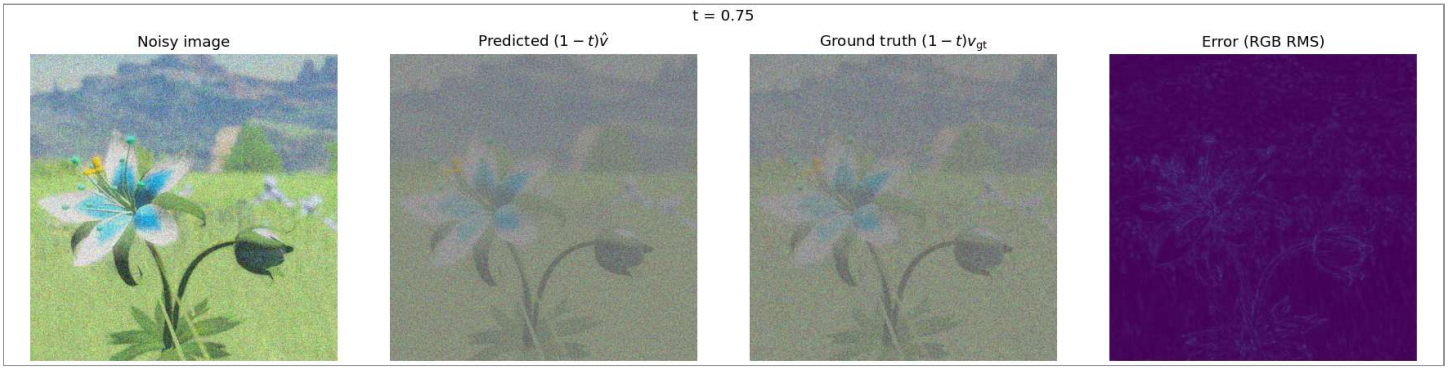


Fig A.1.4d | t = 0.75 | noisy, predicted update, ground-truth update, error

At t = 0.25 the error is brightest on edges and fine detail (stamen tips, petal outlines, bud, leaf edges), and smooth areas like the sky and field are near zero. Both updates look grainy, but the error map has no grain: the predicted and ground-truth noise match so well that the grain cancels in the difference, leaving only differences in image detail.

```
(1 - t)(v_hat - v_gt) = (x1_hat - x_t) - (x1 - x_t) = x1_hat - x1
```

So the error heatmap is exactly the per-pixel error of the A.1.3 one-step estimate, which is why the bright spots match the details A.1.3 lost.

A.1.5

Iterative Denoising with Euler Sampling

`t = 0.39`

I denoised the t = 0.50 input with step h = 0.02 (25 updates), recomputing the velocity each step. The final step is shortened to land exactly on t = 1 (h = min(step_size, 1 - t)), with a 1e-9 tolerance against floating point drift.

```
x_{t+h} = x_t + h * v(x_t, t)
```

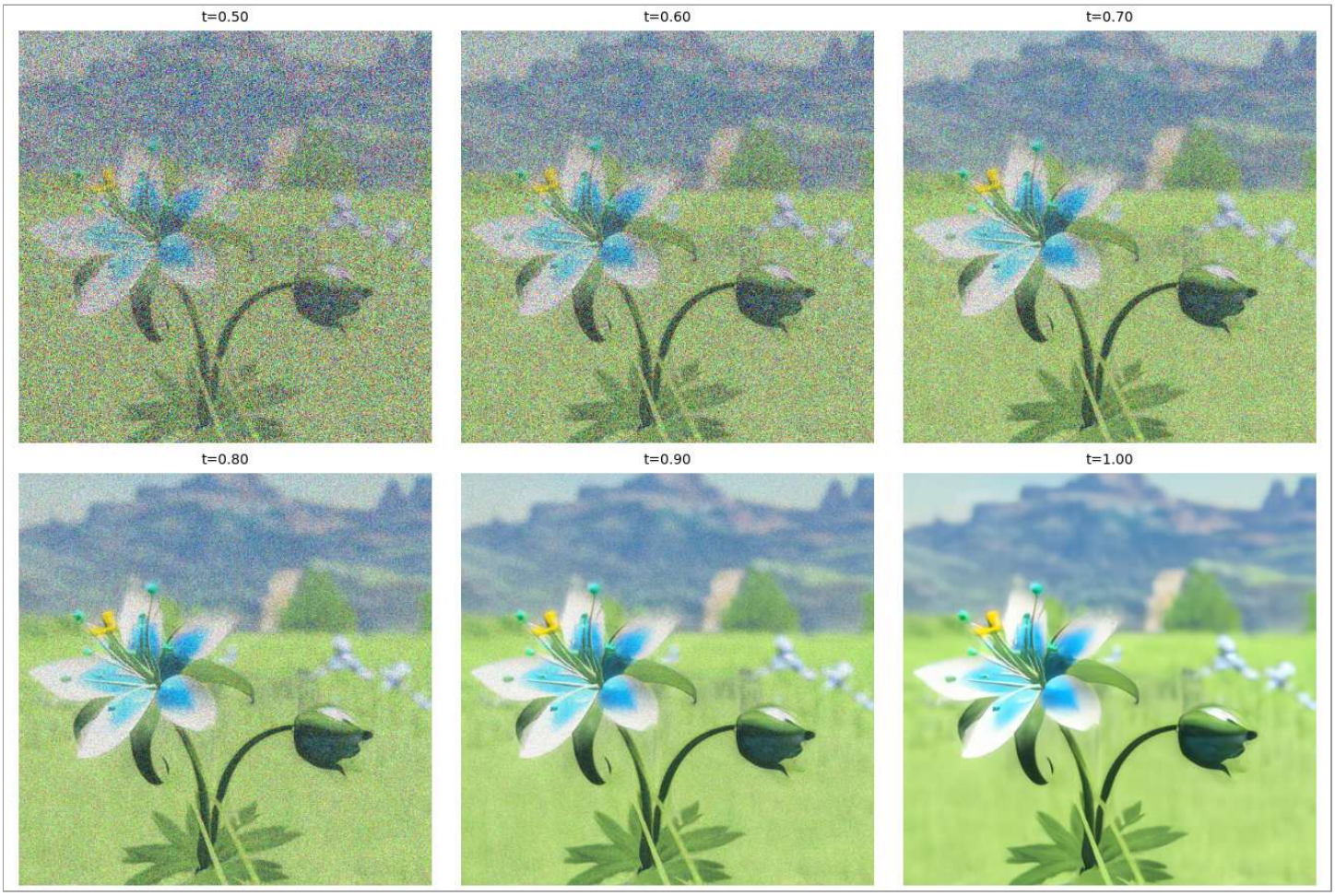


Fig A.1.5a | Euler trajectory | initial t = 0.50, every fifth update, final t = 1.00



Fig A.1.5b | t = 0.50 input | original, blur, one step, Euler

METHOD (T = 0.50 INPUT)	RECONSTRUCTION MSE
Gaussian blur	0.0122
One step	0.0040 (lowest)
Euler, 25 steps	0.0047

Noise fades gradually with each step. At t = 1.00 the teal stamen tips, yellow-orange center, and crisp petal edges are back, all lost in the one-step result. The field has soft vertical streaking and the background flowers differ slightly: sharp and plausible, but not a pixel-perfect copy.

Lowest MSE is not the same as best looking. Euler has sharper detail, but if a detail is slightly off from the original, MSE penalizes it. One step hedges by spreading uncertain details over all plausible positions, so it is never badly wrong anywhere. A sharp detail placed slightly wrong is penalized twice: where it was drawn but shouldn't be, and where it should be but is missing. Squared error rewards blurry averages over committing to a sharp guess.

A.1.6

Generating Images from Noise

`t = 0.46`

Five independent Gaussian starts (seeds 180 to 184) integrated from t = 0 to 1 with 50 Euler updates and the prompt `"a high quality photo"`.

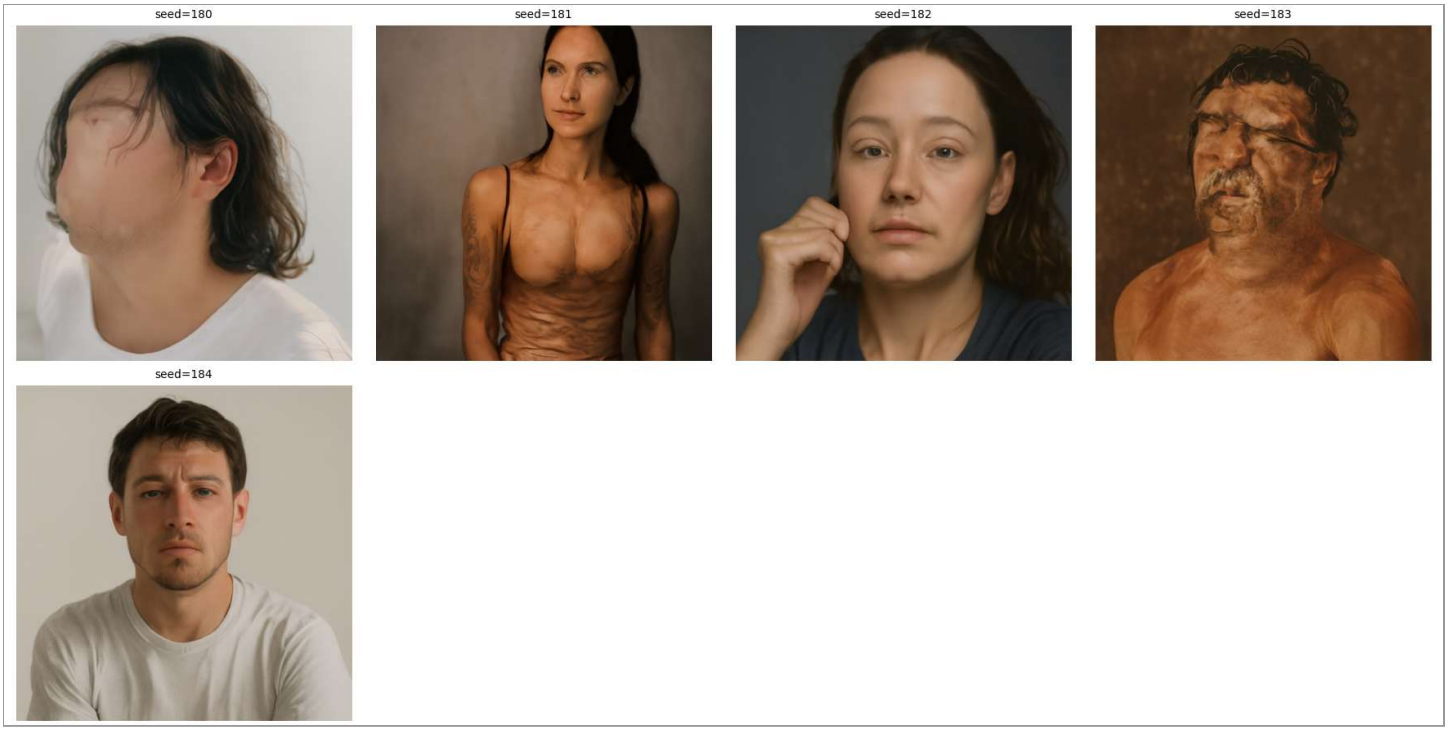


Fig A.1.6 | no guidance | "a high quality photo" | seeds 180, 181, 182, 183, 184

All five seeds produced portraits of people, even though the prompt says nothing about people. Seeds 182 and 184 are plausible, coherent portraits. Seed 181 has a believable face, but the torso has melted skin texture. Seeds 180 and 183 have badly distorted faces (180 is missing features, 183 is melted).

My hypothesis is that a common type of high quality image is a professional portrait shot with one clear subject, so the prompt favors portraits. Many keep only the general shape, silhouette, and color of a person. These samples are the baseline for A.2.1, which reuses the same starting noises.

A.2.1

Classifier-Free Guidance

```
t = 0.54
```

Both branches are evaluated at the same x and t, with an empty-string null condition. w = 0 gives the ordinary conditional prediction; higher w overcorrects toward the prompt. I used w = 7, h = 0.02, and the same seeds as A.1.6, so the rows compare seed by seed.

```
v_cfg = v_c + w * (v_c - v_u)
```

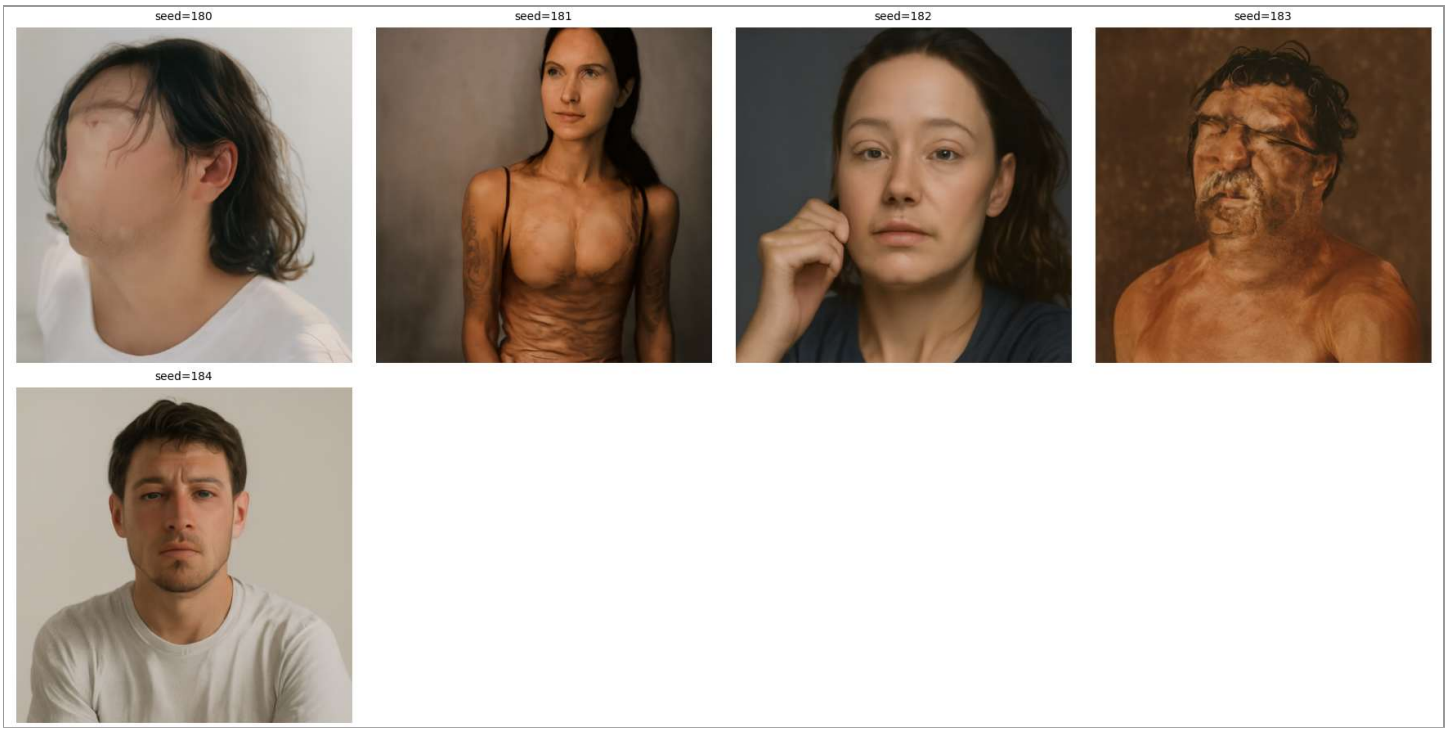


Fig A.2.1a | without CFG (A.1.6, repeated for comparison) | seeds 180 to 184

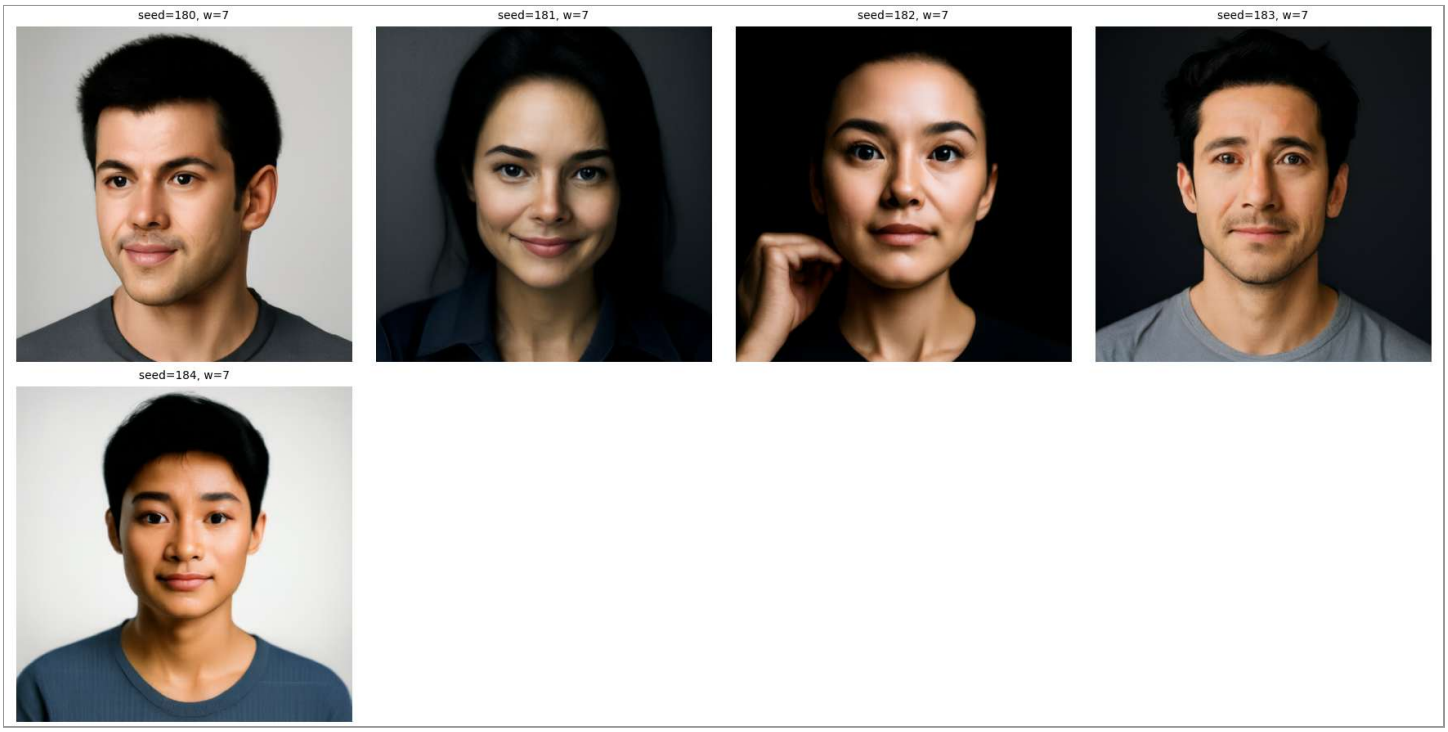


Fig A.2.1b | CFG w = 7 | "a high quality photo" | seeds 180 to 184

Seeds 180 and 183 went from melted, featureless faces to clean, coherent portraits, and seed 181 lost the odd torso texture. The faces are much more human-like, neutral or smiling, looking at the camera, with detail like wrinkles. Contrast is higher, with more dramatic lighting and several dark studio backgrounds. Seed 182 kept the hand raised to the face, so the same starting noise keeps the rough composition.

But there is no variety: similar smiles, hair color, eye color, and position in the frame. Guidance exaggerates the prompt's push, so every sample is driven toward the model's single most typical "high quality photo." Variety is the sacrifice for image quality.

A.3.1

Image-to-Image Editing

```
t = 0.61
```

To edit, I build the start point with `interpolate(image, noise, t_start)` and integrate from t_start to 1 with Euler sampling and CFG (`"a high quality photo"`, w = 7, h = 0.02, the same noise for every t and image). The velocity is passed as a lambda of (x, t) so it is recomputed each step. Each sweep reads left to right, top to bottom, from uncanny to recognizable.

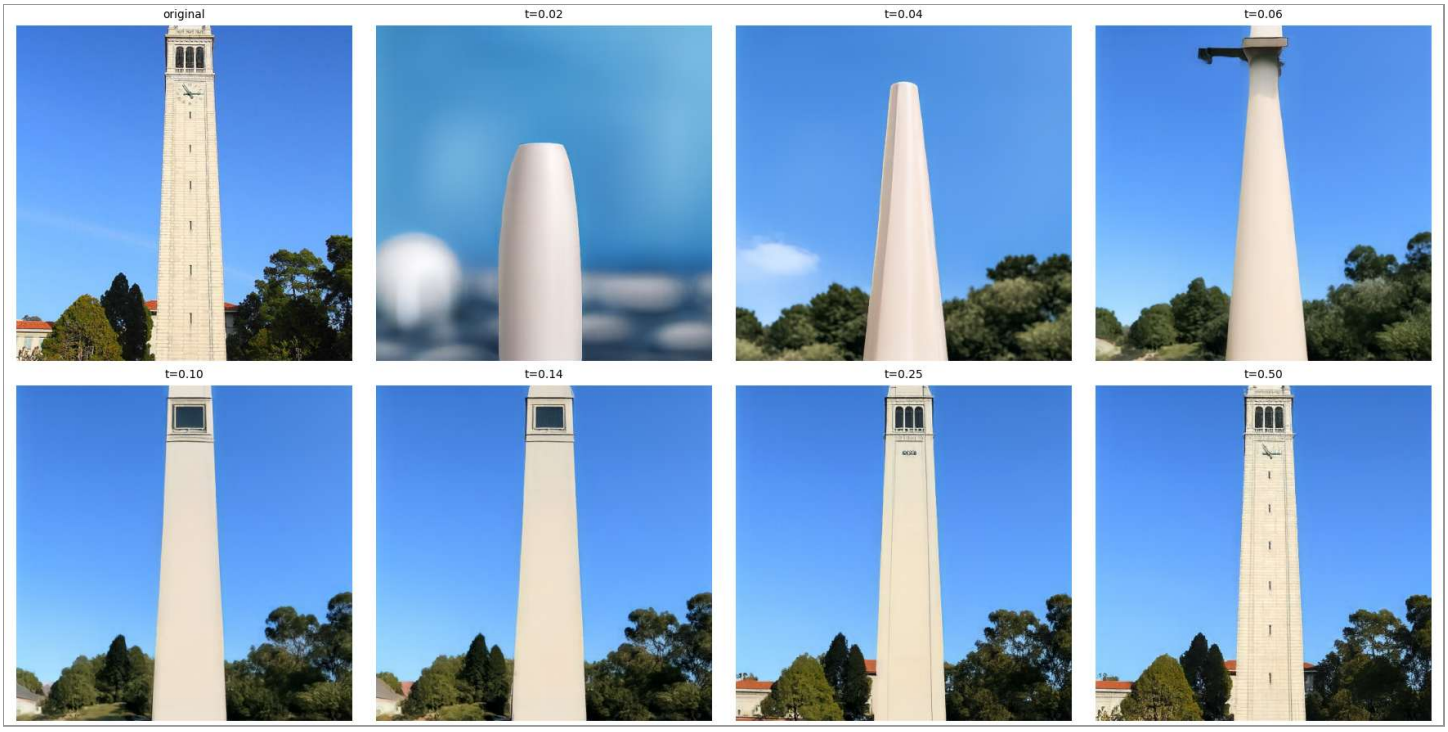


Fig A.3.1a | Campanile | "a high quality photo" | t = 0.02 to 0.50



Fig A.3.1b | Silent Princess (game render) | "a high quality photo" | t = 0.02 to 0.50

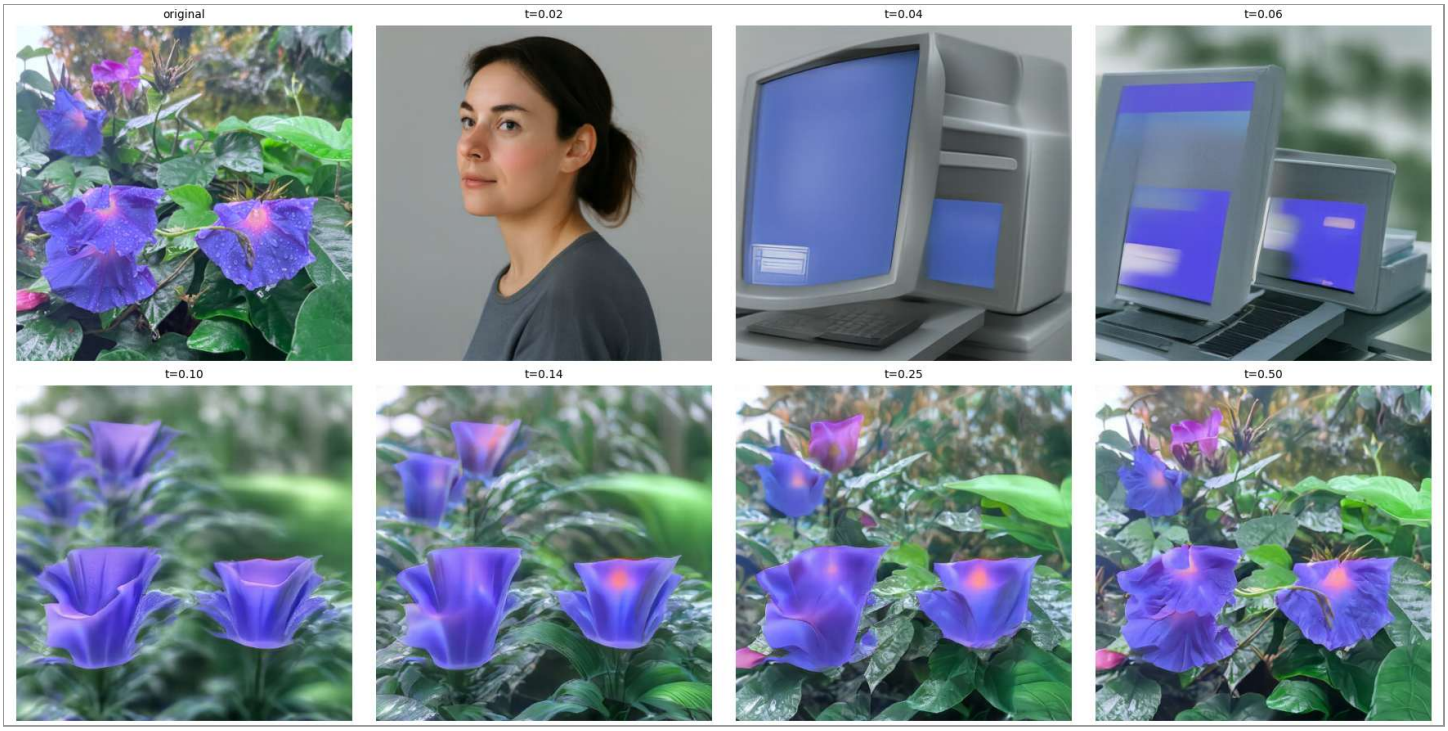


Fig A.3.1c | Morning glory (my photo) | "a high quality photo" | t = 0.02 to 0.50

General pattern

- **t = 0.02 to 0.06:** only color palette and rough layout survive; the model invents a new scene in those colors.
- **t = 0.10 to 0.14:** the subject comes back, simplified.
- **t = 0.25:** subject plus mid-level detail.
- **t = 0.50:** very close to the original, including fine detail.

Campanile: even at 0.02, a tall white shape against blue sky survives; a strong, simple vertical layout is hard to lose. At 0.10 to 0.14 it becomes a plain obelisk, 0.25 has windows, and 0.50 has the clock.

Silent Princess: low t became a picture frame, a magazine, and a phone in the original's blue, white, and green. Low t pulls toward screens, frames, and devices: since only color and layout survive, a blue focus in the center can look like a glowing screen. My hypothesis is that product photos of devices are very common online and often described as "high quality." The game render changed more across the sweep: unrecognizable from 0.02 to 0.06, with odd squarish, screen-like artifacts at 0.10 to 0.14, and close to the original only at 0.25. The model learned from real photos, so the stylized render is less familiar and it keeps trying to "correct" it.

Morning glory: at 0.02 a portrait of a person appears, and at 0.04 to 0.06 computer monitors in the flowers' purple-blue. The sweet spot is t = 0.10, where the model starts generating flowers instead of screens. At 0.10 they are general flowers, not specifically mine; at 0.14 details appear that make them more like my flowers.

A.3.2

Text-Guided Image Editing

`t = 0.68`

Same three inputs and seven start times as A.3.1, with a new prompt embedding in place of "a high quality photo."

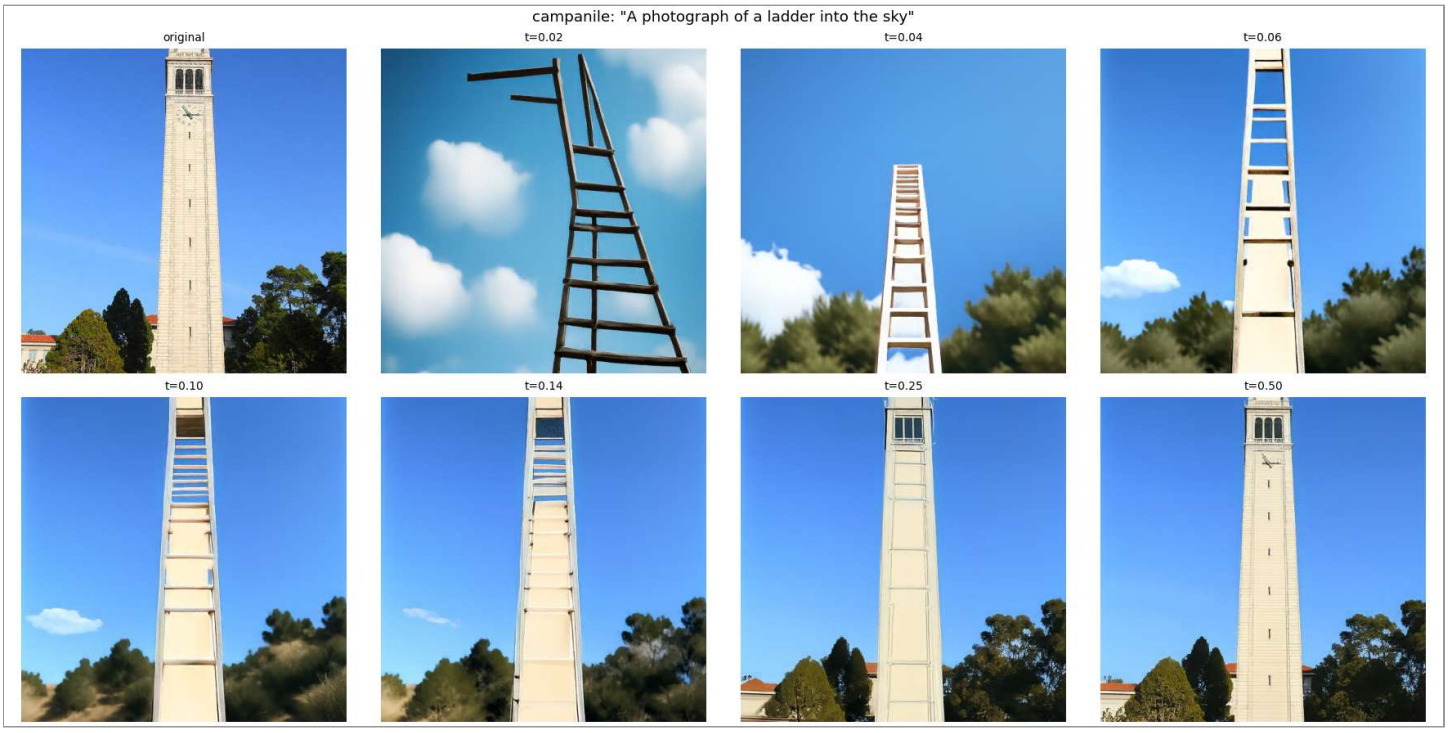


Fig A.3.2a | Campanile | "A photograph of a ladder into the sky" | t = 0.02 to 0.50

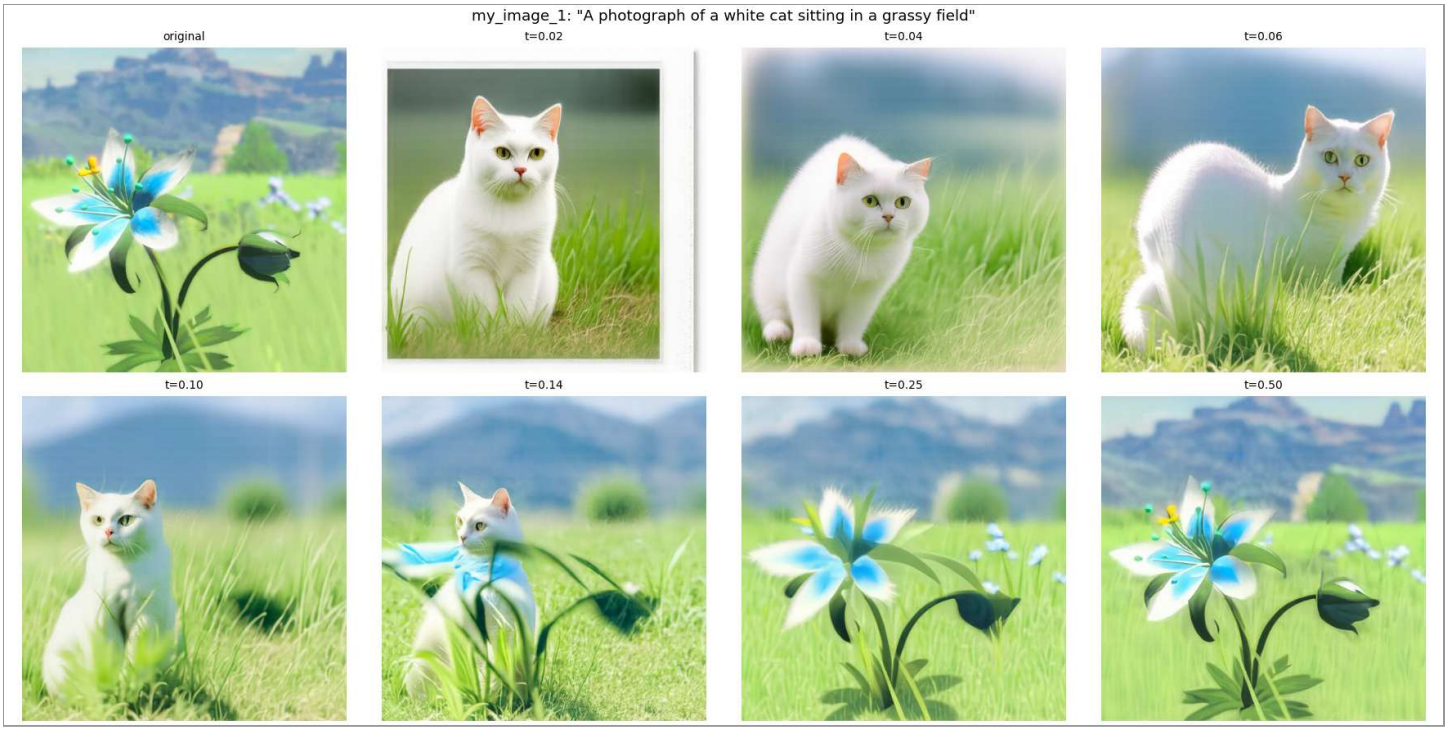


Fig A.3.2b | Silent Princess | "A photograph of a white cat sitting in a grassy field" | t = 0.02 to 0.50

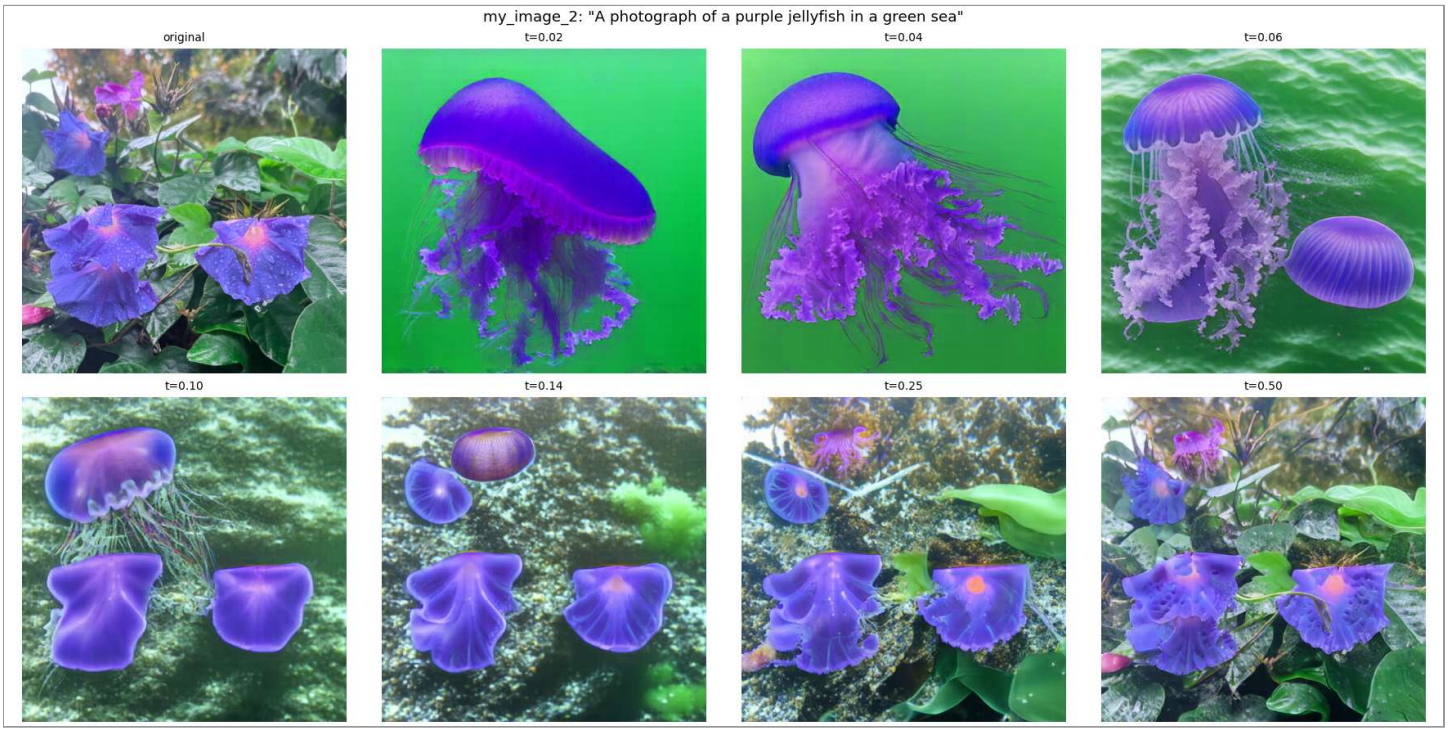


Fig A.3.2c | Morning glory | "A photograph of a purple jellyfish in a green sea" | t = 0.02 to 0.50

- **Ladder:** the prompt wins through t = 0.14, with the ladder inheriting the tower's taper and position; the Campanile takes over at 0.25.

- **Cat:** holds through 0.14 (at 0.14 a hybrid, the cat wears the flower's blue petals like a scarf); the flower returns at 0.25.
- **Jellyfish:** holds the prompt all the way to t = 0.50, with the original leaves around the jellyfish.

It comes down to how well the prompt matches the original's shape and color. Purple bell-shaped blooms already look like jellyfish, so prompt and image agree at every t. The ladder and cat are entirely new subjects that fight the original's shapes, so the original wins once enough of it is kept.

A.3.3

Editing Hand-Drawn and Web Images

`t = 0.75`

Inputs: a web image (dewy white lily stock photo), a crewmate drawing (red color fill), and a cat drawing (line art on lined paper). The drawings were cropped square with paper-colored padding. Same settings and prompt as A.3.1.

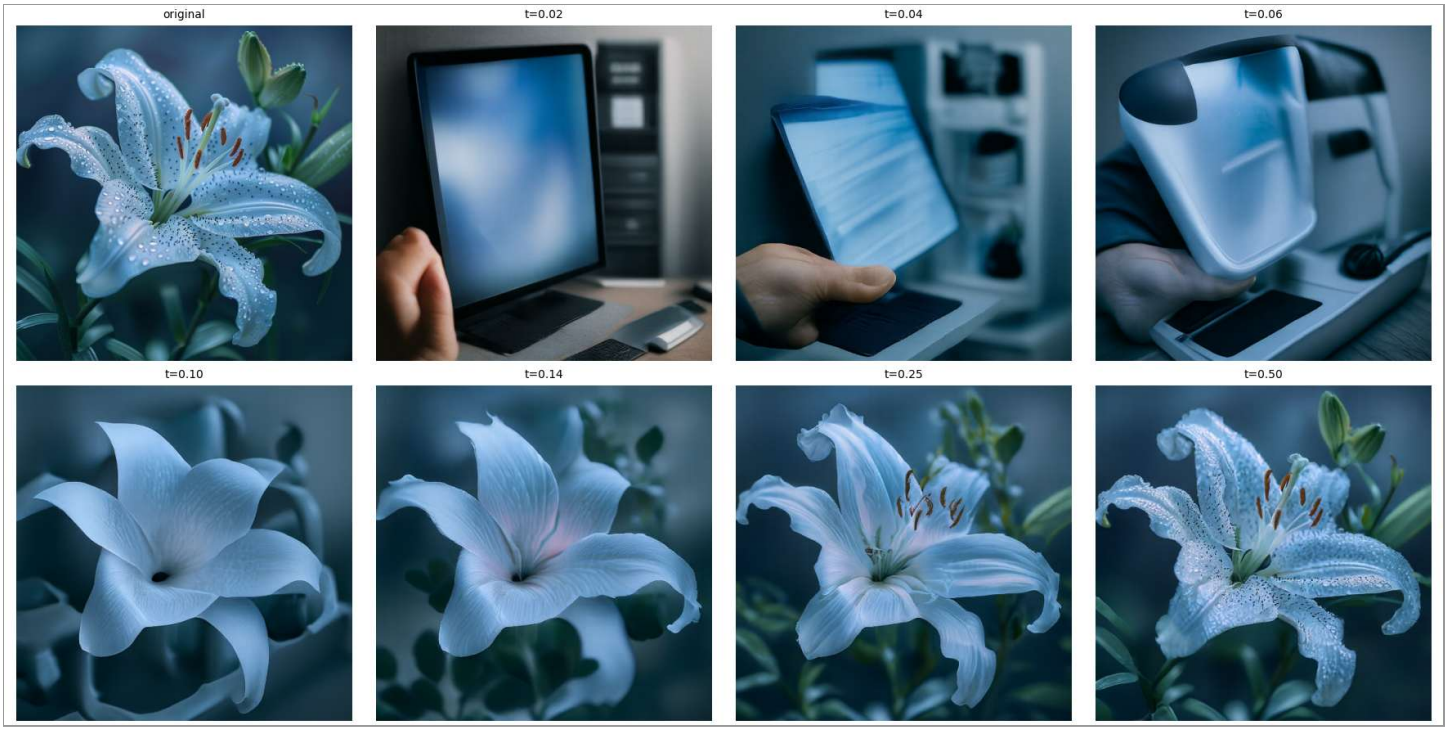


Fig A.3.3a | Lily (web image) | "a high quality photo" | t = 0.02 to 0.50

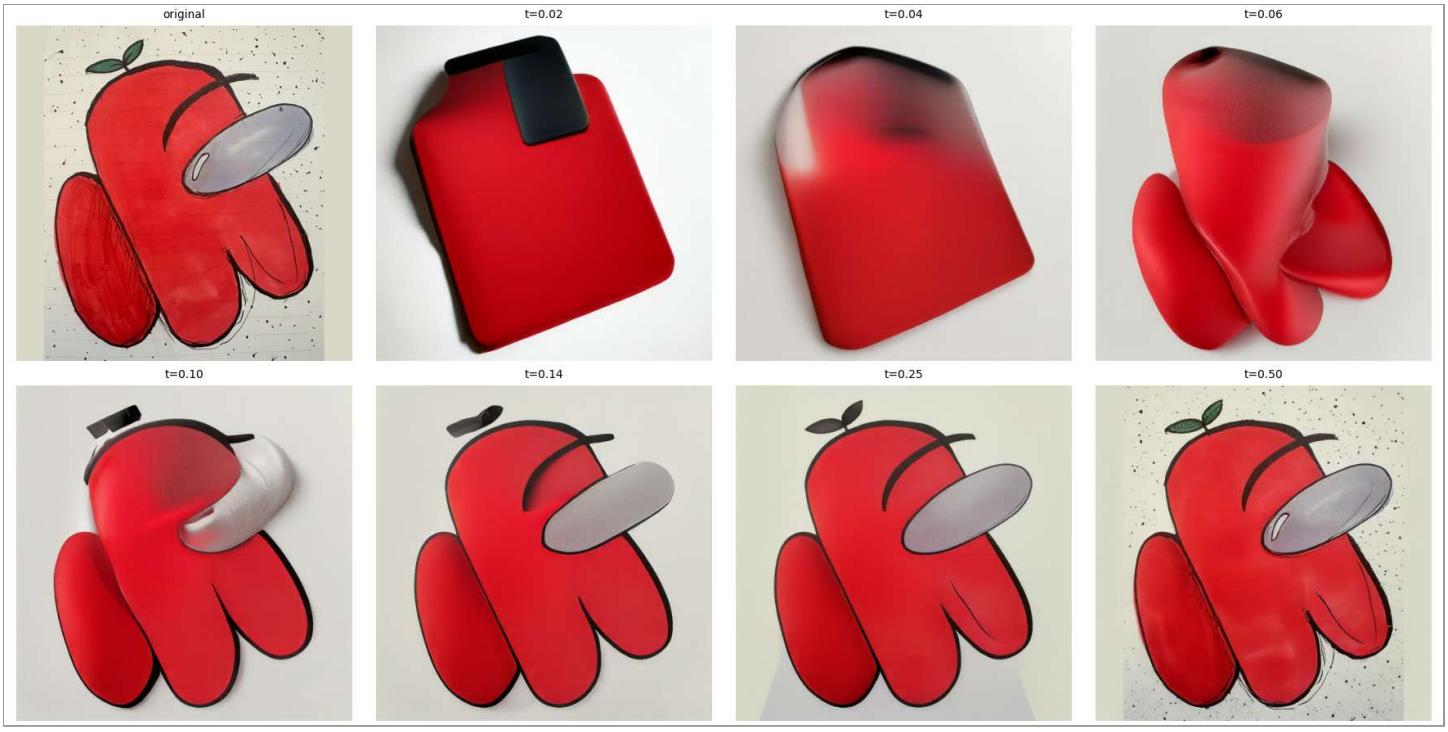


Fig A.3.3b | Crewmate drawing | "a high quality photo" | t = 0.02 to 0.50

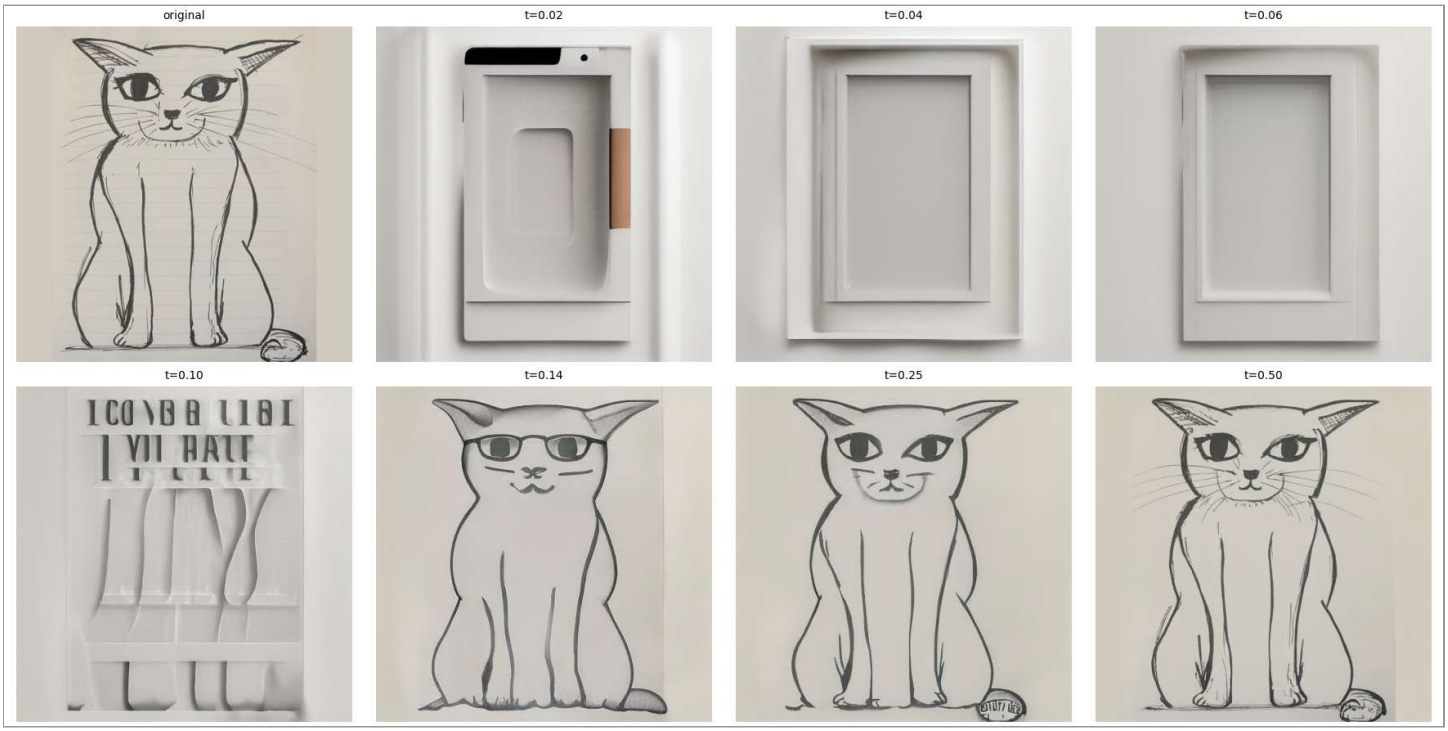


Fig A.3.3c | Cat drawing | "a high quality photo" | t = 0.02 to 0.50

Lily: at 0.02 to 0.06 a computer monitor and devices, with a human hand in all three (connecting to the people bias from A.1.6 and A.2.1). At 0.10 to 0.14 a clean, simplified lily with no stamens or droplets; stamens return at 0.25, and water droplets and speckles at 0.50. It moved cleanly from desk scene to flower at 0.10, unlike the Silent Princess render.

Crewmate: low t became red objects (a phone case or tablet), then a glossy 3D figure at 0.06 to 0.10, and back to a drawing by 0.14. It kept its identity at lower t thanks to big features: one bold red color and a rounded rectangle shape.

Cat: low t became picture frames and devices, at 0.10 it hallucinated fake text, and at 0.14 the cat got glasses. Its thin lines can look like thin shadows, so an image made of small details only comes back at higher t. The text at 0.10 appears because thin lines on lined paper resemble where writing would be; photos of handwritten lined pages are likely very common.

Drawings stay drawings. At higher t the starting image fixes not just layout but medium and texture (paper, ink, flat color), and "a high quality photo" isn't strong enough to override it. Overall, the more photo-like the input, the smoother the transition back to the original.

A.3.4

Visual Anagrams

`t = 0.83`

Each step averages the velocity for prompt A with the velocity for prompt B computed on the flipped image. The B velocity must be rotated back before averaging.

```
v = 0.5 * ( v_cfg(x, t, c_a) + rot180( v_cfg(rot180(x), t, c_b) ) )
```

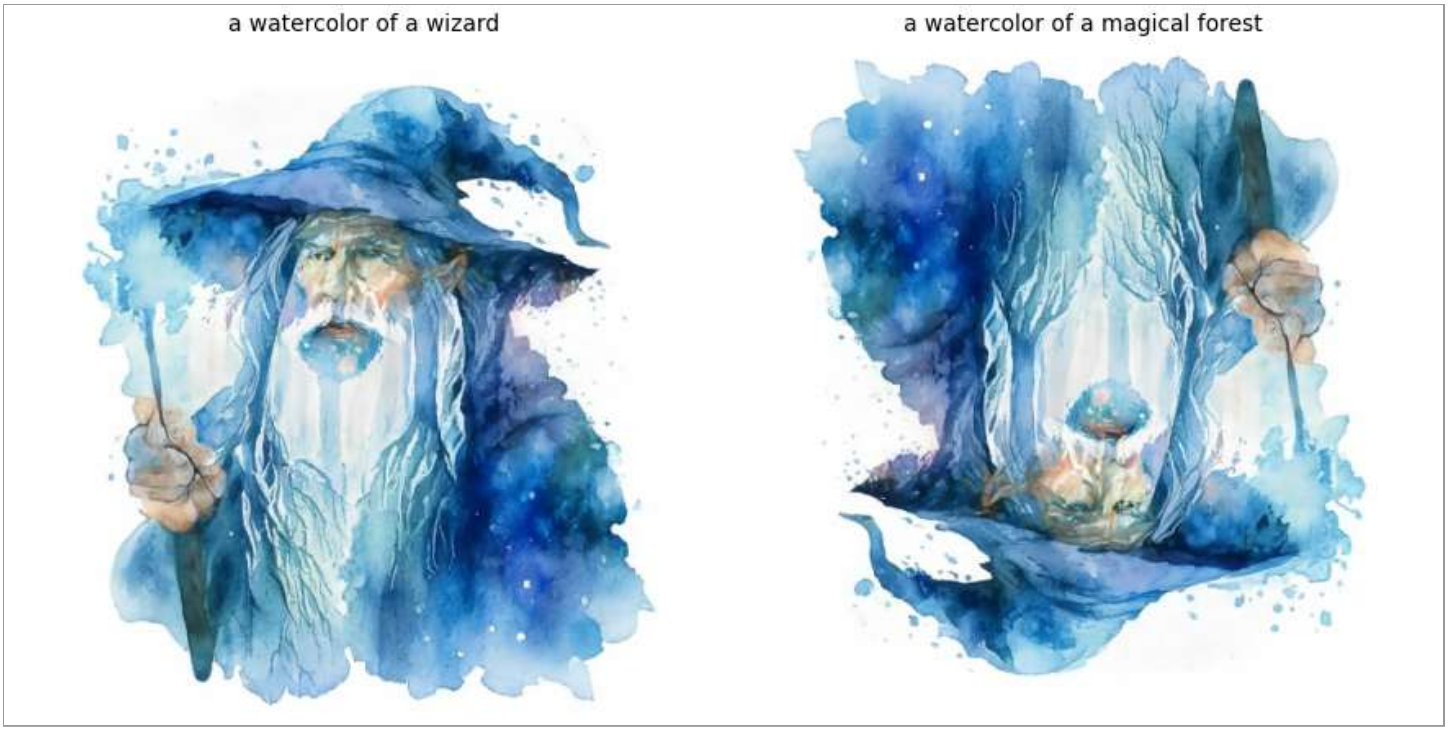


Fig A.3.4a | upright: "a watercolor of a wizard" | rotated 180: "a watercolor of a magical forest"



Fig A.3.4b | upright: "an oil painting of a teapot" | rotated 180: "an oil painting of a bird"

A.3.5

Hybrid Images

```
t = 0.90
```

The low frequencies of one prompt's velocity are combined with the high frequencies of the other. Prompt a sets the distant structure and prompt b the close-up detail. Viewed at full size you see the close-up prompt; at small size, the distant one.

```
v = lowpass(v_a) + v_b - lowpass(v_b)
```

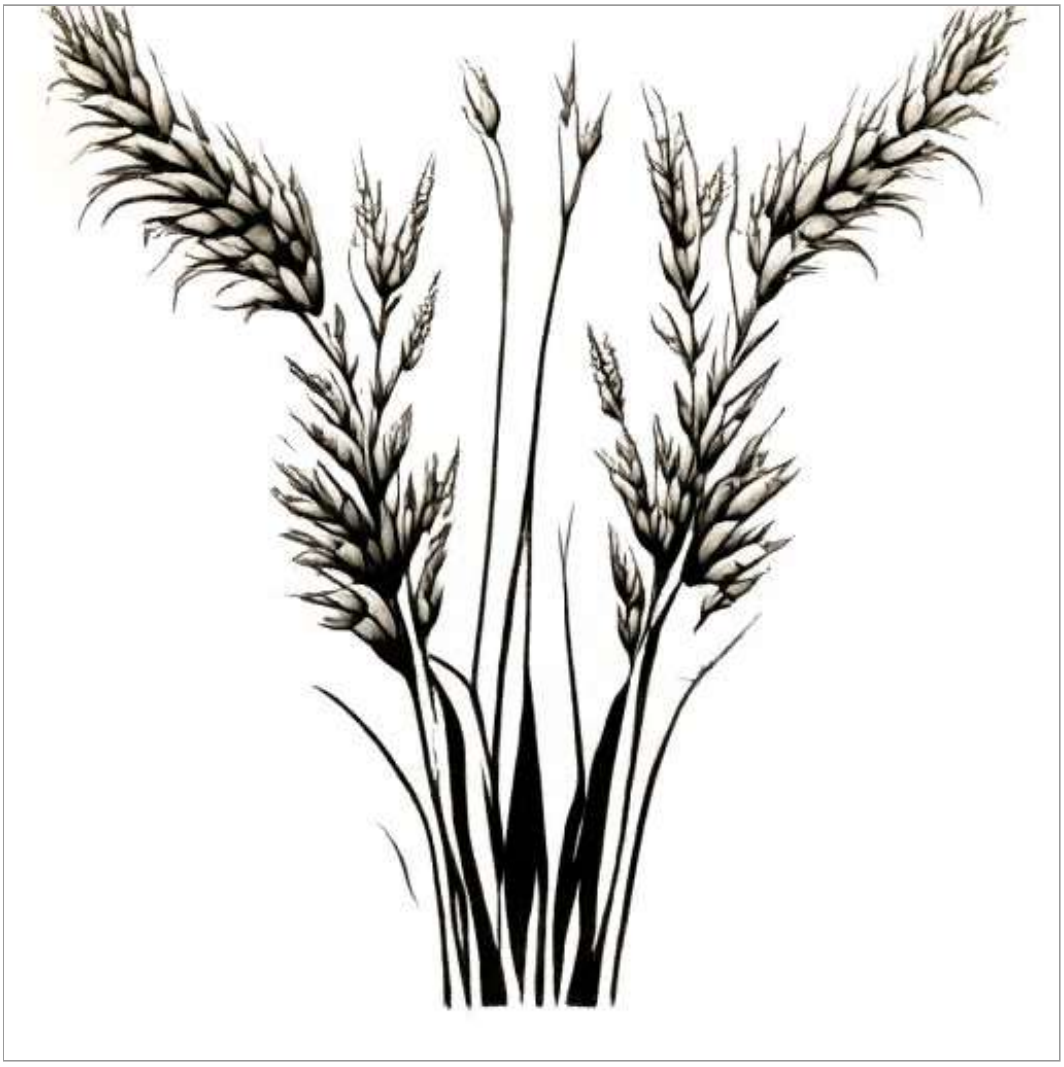




Fig A.3.5a | full size | close-up: wheat field | distant: lion face

reduced size | lion face





Fig A.3.5b | full size | close-up: erupting volcano | distant: dragon head

reduced size | dragon head

Part B

Flow Matching from Scratch

`t = 0.91 to 0.98`

`B.1.1`

UNet

`t = 0.91`

I built the simple ops (Conv, DownConv, UpConv, Flatten, Unflatten) and the composed ops (ConvBlock, DownBlock, UpBlock), all shape-tested, then assembled the UnconditionalUNet. The forward pass saves x1, x2, x3 for the three skip concatenations (`torch.cat` on dim = 1). An input of shape (4, 1, 28, 28) returns (4, 1, 28, 28).

`B.1.2`

Noising Process

`t = 0.92`

```
z = x + sigma * eps, sigma in {0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0}
```

Unlike Part A, the digit is never scaled down; noise is piled on top.

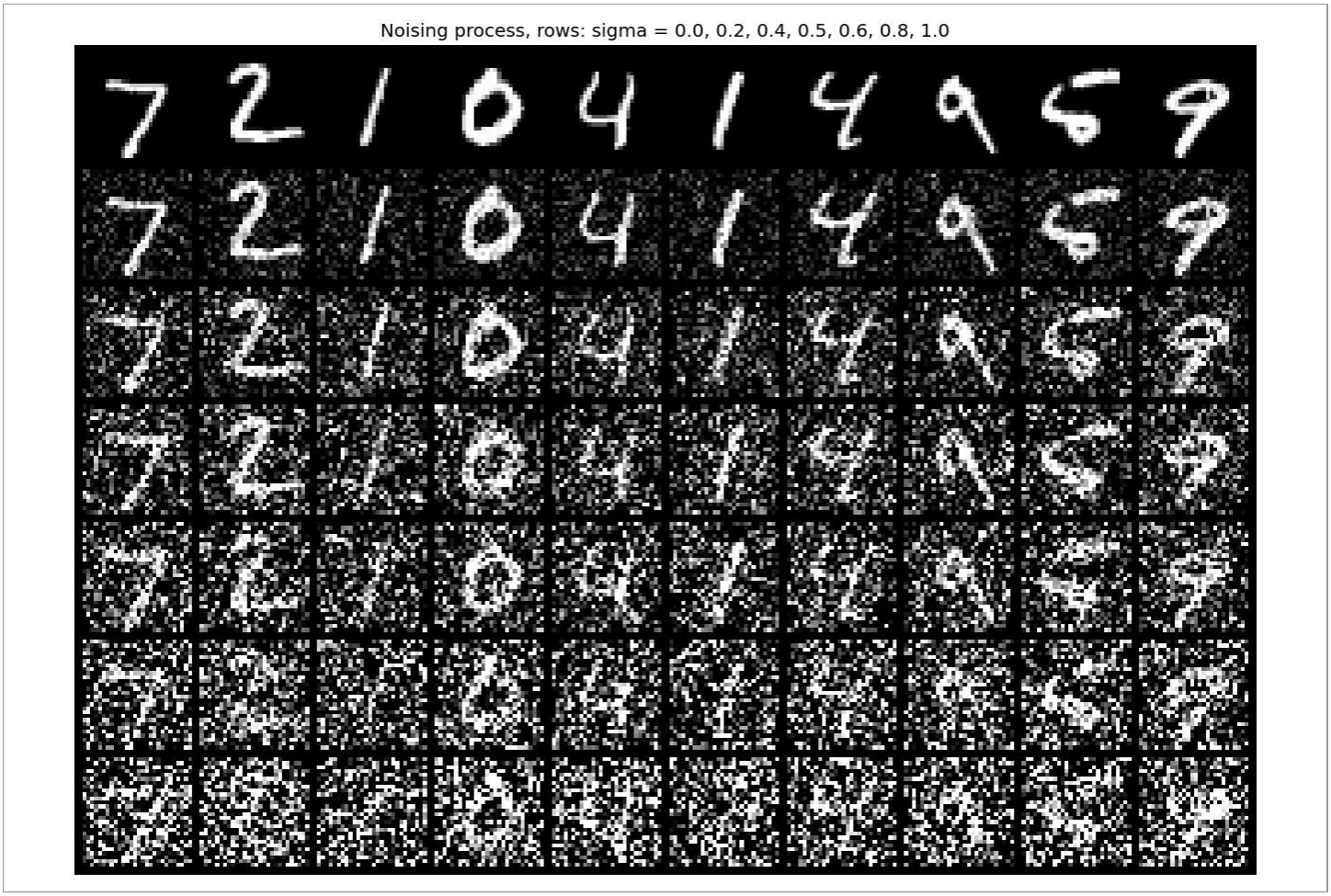


Fig B.1.2 | MNIST digits | sigma = 0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0

Digits are easy to read through sigma = 0.4, rough at 0.5 to 0.6, and mostly unreadable at 0.8 to 1.0. The 0 and 9 survive longest.

B.1.2.1

Training the Denoiser

```
t = 0.93
```

Trained at sigma = 0.5 with D = 128, Adam lr = 1e-4, batch size 256, for 5 epochs (about 235 steps per epoch, 1,175 total). The clean images stay the target.

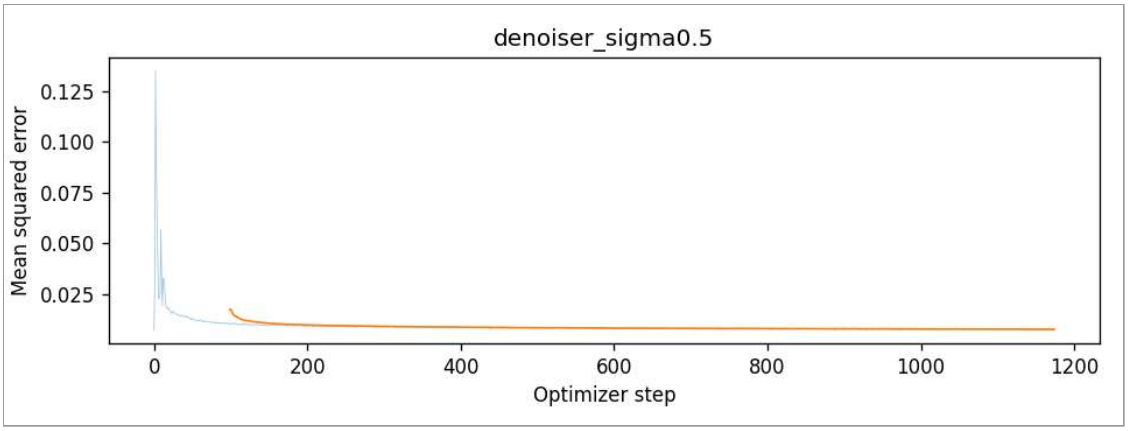


Fig B.1.2.1a | training loss | sigma = 0.5 | 5 epochs

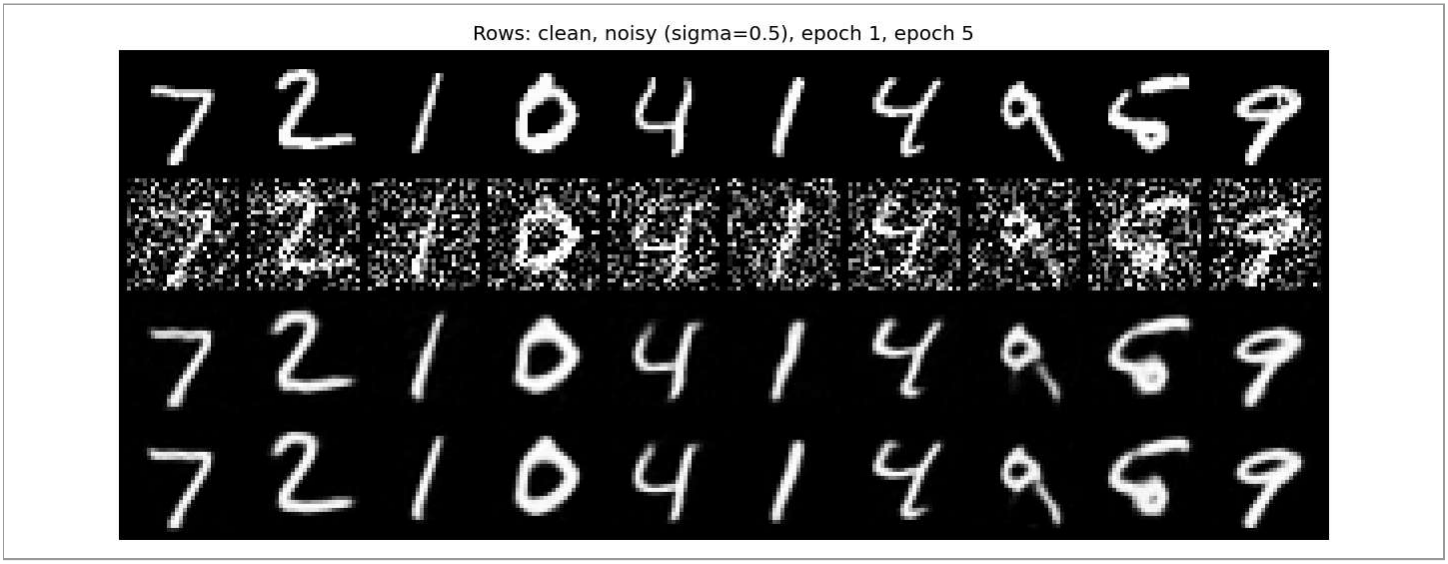


Fig B.1.2.1b | clean, noisy (sigma = 0.5), denoised after epoch 1, denoised after epoch 5

The loss starts near 0.13 and drops sharply in the first 50 steps (learning the black background), then declines slowly to about 0.008 (refining stroke shapes). Both epoch 1 and epoch 5 outputs are clean; epoch 5 is slightly sharper.

B.1.2.2

Out-of-Distribution Testing

```
t = 0.94
```

The same 10 test digits at sigma from 0.0 to 1.0, with the model trained only at sigma = 0.5.

OOD test: rows alternate noisy / denoised, sigma = 0.0 to 1.0

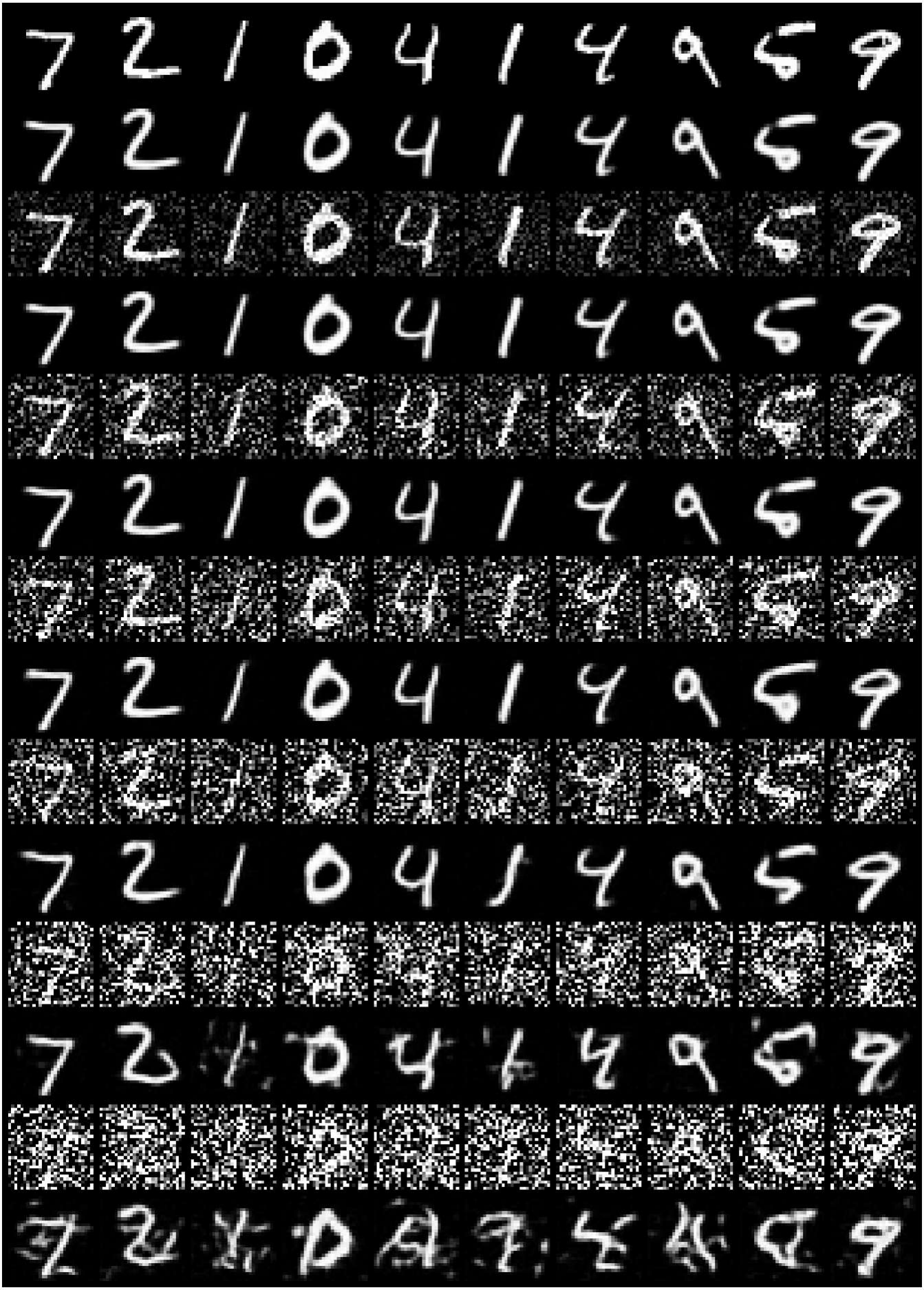


Fig B.1.2.2 | noisy and denoised rows | sigma = 0.0 to 1.0 | trained at sigma = 0.5

- **sigma = 0.0 to 0.5:** nearly perfect; at 0.0 the digits are left basically untouched.
- **sigma = 0.6:** first small artifacts.
- **sigma = 0.8:** strokes break up, with stray speckles.
- **sigma = 1.0:** many digits fall apart (the 1 becomes a squiggle, the 2 becomes "21"); the 0, 4, and 9 survive.

B.1.2.3

Denoising Pure Noise

```
t = 0.95
```

Same setup, but the input is pure noise z = eps while the target is still the clean digit.

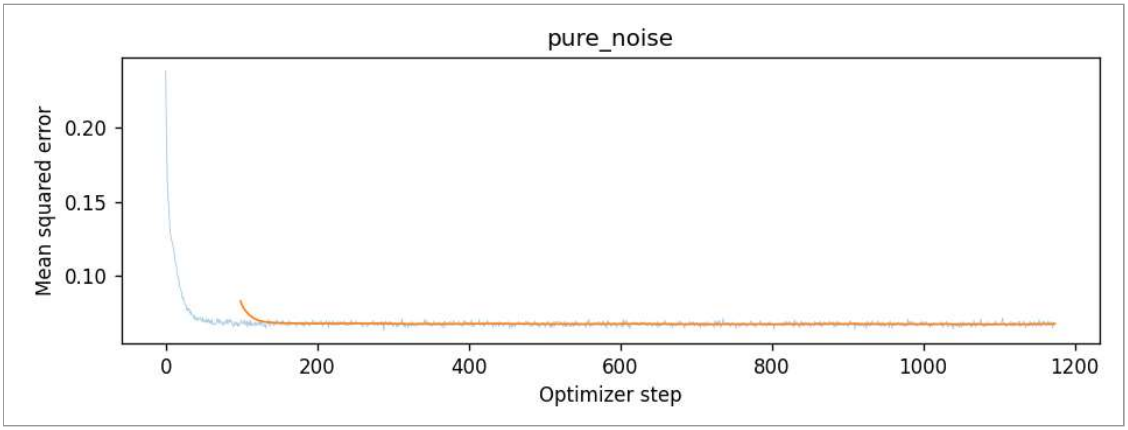


Fig B.1.2.3a | training loss | input z = eps

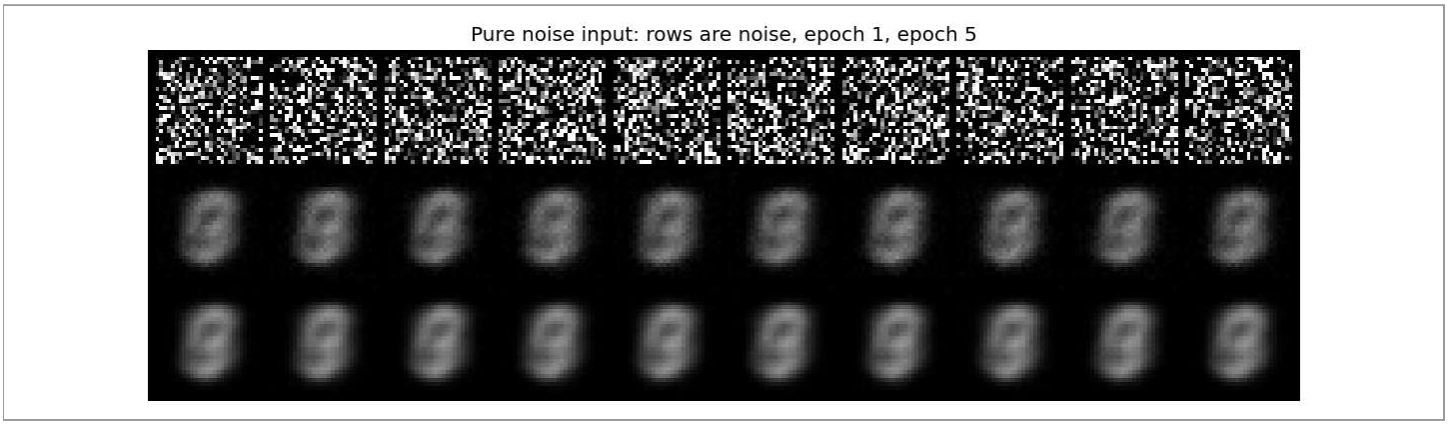


Fig B.1.2.3b | pure noise input, output after epoch 1, output after epoch 5

Every output is the same blurry image regardless of input, like a 0, 3, 8, and 9 overlapped; epoch 5 is slightly smoother. The loss drops fast within about 100 steps and then sits flat at about 0.068 (versus 0.008 for the B.1.2.1 denoiser).

The model averages all the digits together and outputs the mean of the data, roughly the pixel-by-pixel average of all ten digits. Strokes many digits share (the center loop area) are brightest; strokes unique to one digit (the 7's top bar, the 1's thin line) fade. The noise carries no information about the target, so the model can only output one fixed image, and under MSE the image with the lowest average squared error against all training digits is their mean. The flat 0.068 floor is how different real digits are from their average; no amount of training removes it. Same idea as Part A: MSE rewards blurry averages.

B.2.3

Time-Conditioned Flow Matching

```
t = 0.96
```

An FCBlock (Linear, GELU, Linear) embeds t: fc1_t (1 to 2D) modulates the unflatten output and fc2_t (1 to D) modulates the up1 output. Training samples t uniformly per image and regresses the velocity; sampling runs 50 Euler steps from noise. D = 64, batch 64, Adam lr 1e-2, ExponentialLR gamma = 0.1^(1/10), 10 epochs.

```
x_t = (1 - t) * x_0 + t * x_1, target = x_1 - x_0, loss = MSE
```

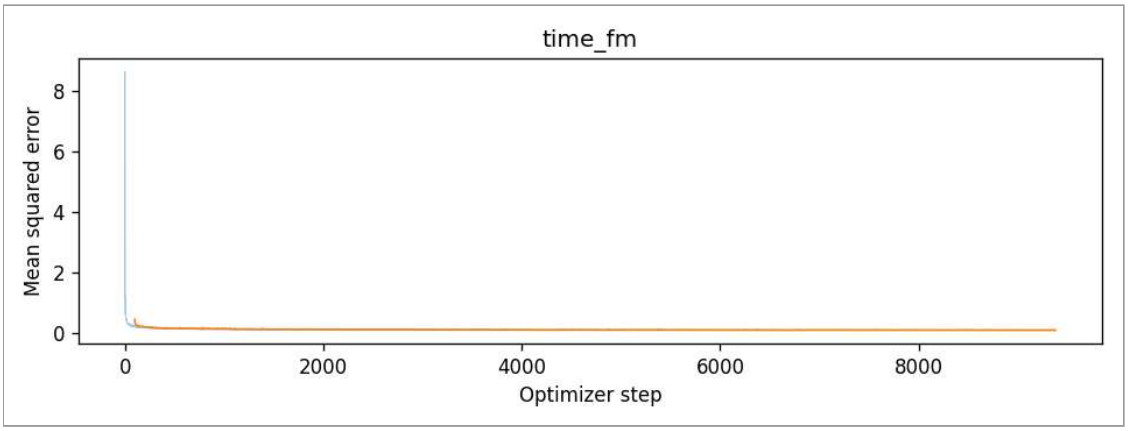


Fig B.2.3a | training loss | time-conditioned UNet | 10 epochs

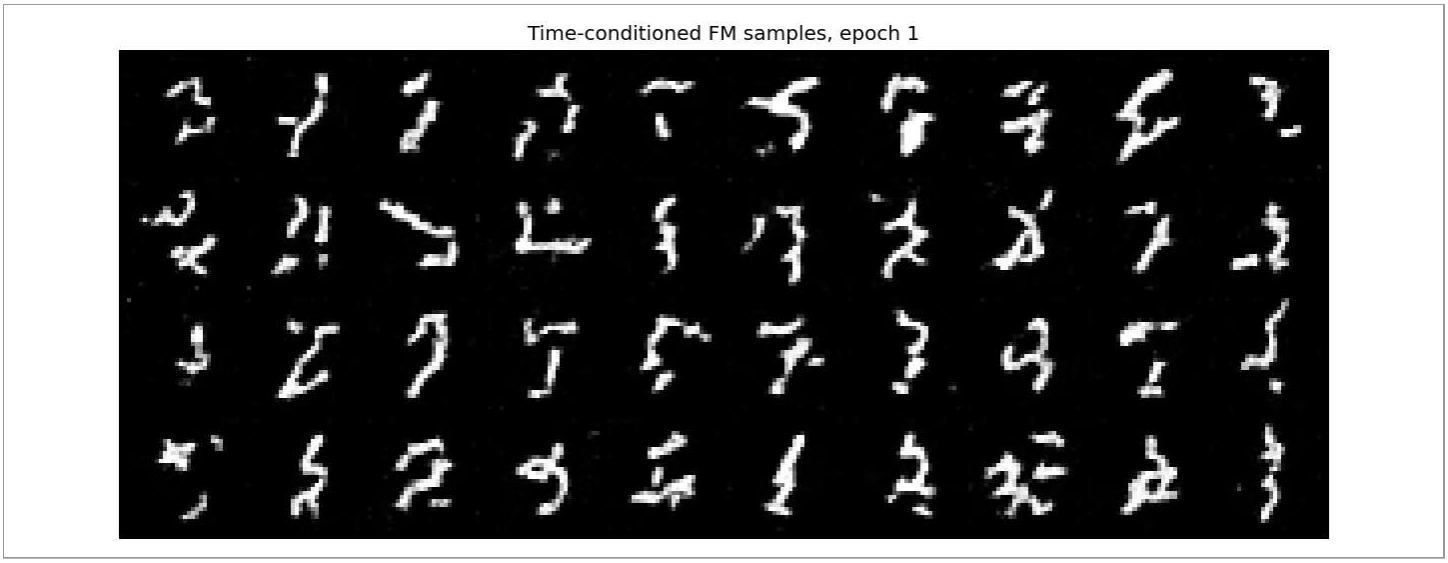


Fig B.2.3b | samples | epoch 1

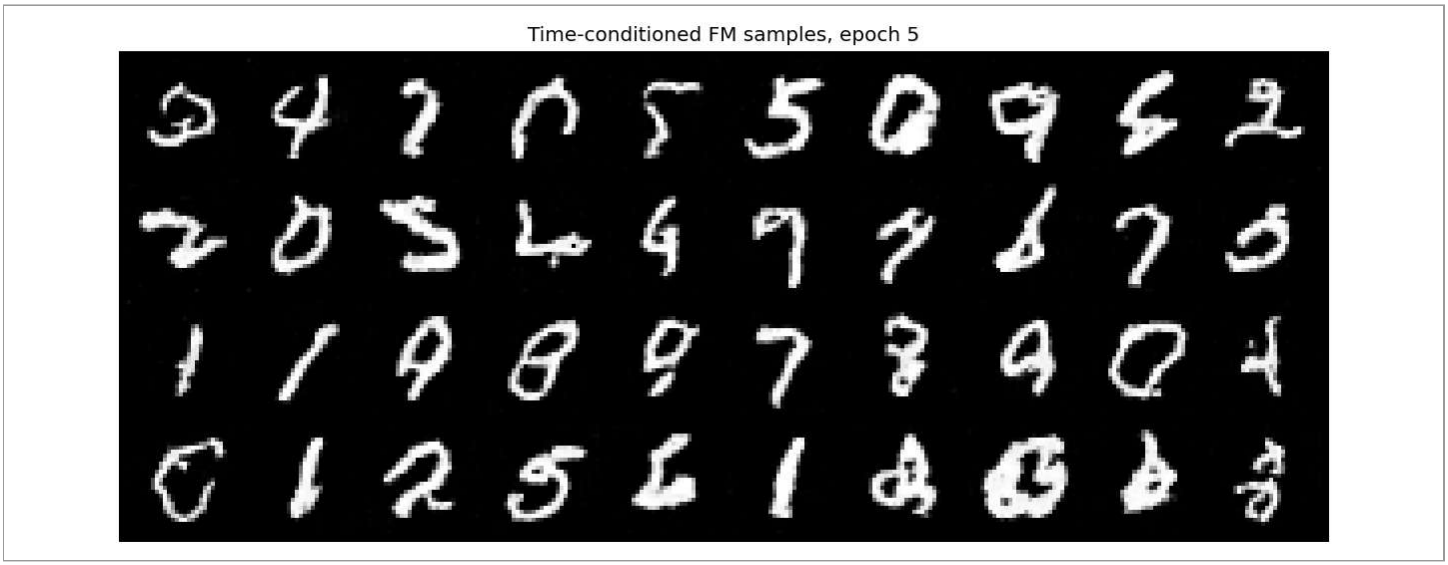


Fig B.2.3c | samples | epoch 5

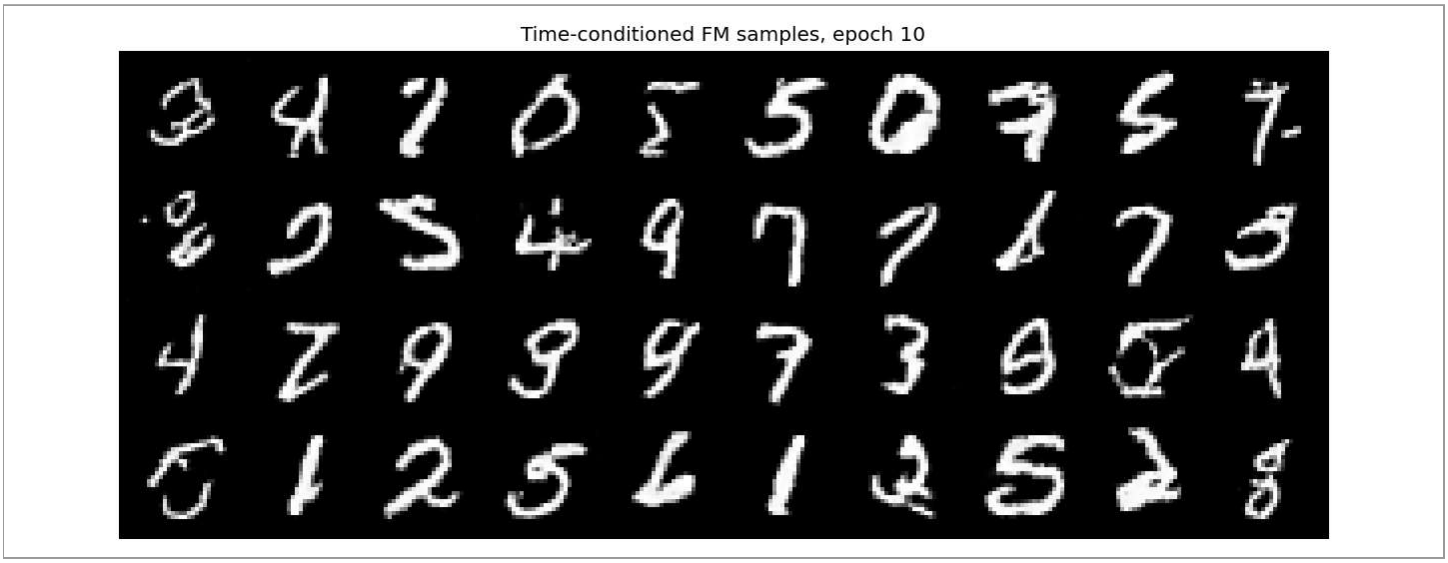


Fig B.2.3d | samples | epoch 10

- **Epoch 1:** stroke-like squiggles; it learned "white strokes on black" but not digit shapes.
- **Epoch 5:** most samples are recognizable digits, some malformed.
- **Epoch 10:** cleaner, more confident strokes; a few hybrids remain.

The samples are a mix of digits since there is no class information. With the same seed at every checkpoint, many positions keep the same digit from epoch 5 to 10: training sharpens them rather than changing them.

B.2.6

Class-Conditioned Flow Matching

```
t = 0.98
```

The class is a one-hot vector with 10% class dropout. fc1_c (10 to 2D) and fc2_c (10 to D) scale the features while time shifts them: unflatten = c1 * unflatten + t1, up1 = c2 * up1 + t2. Sampling uses CFG with gamma = 5 and 300 Euler steps, 4 of each digit.

```
v = v_u + gamma * (v_c - v_u), gamma = 5
```

With scheduler (lr 1e-2, exponential decay)

No scheduler (constant lr 2e-3)

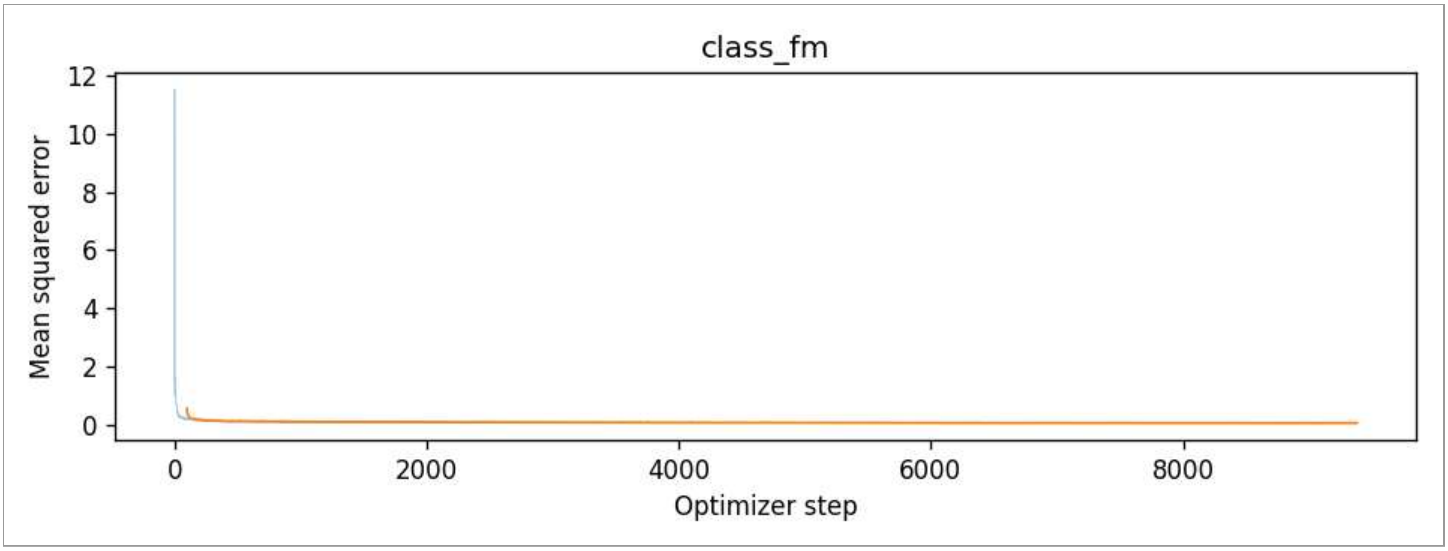


Fig B.2.6a | scheduler | training loss

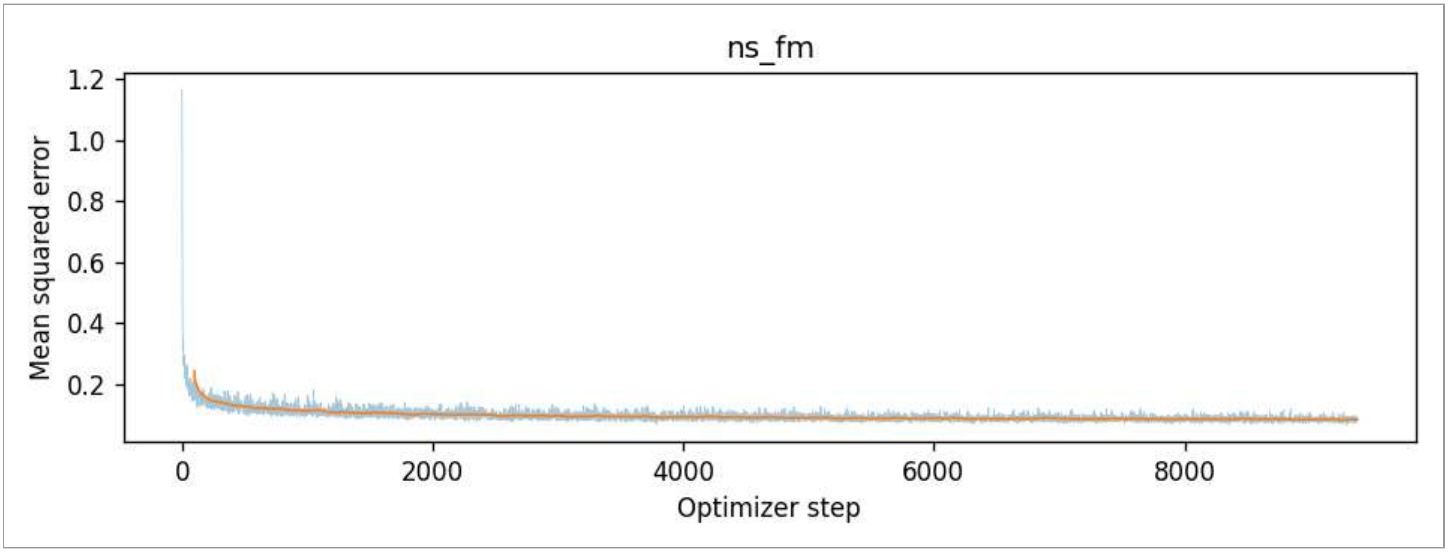


Fig B.2.6b | no scheduler | training loss

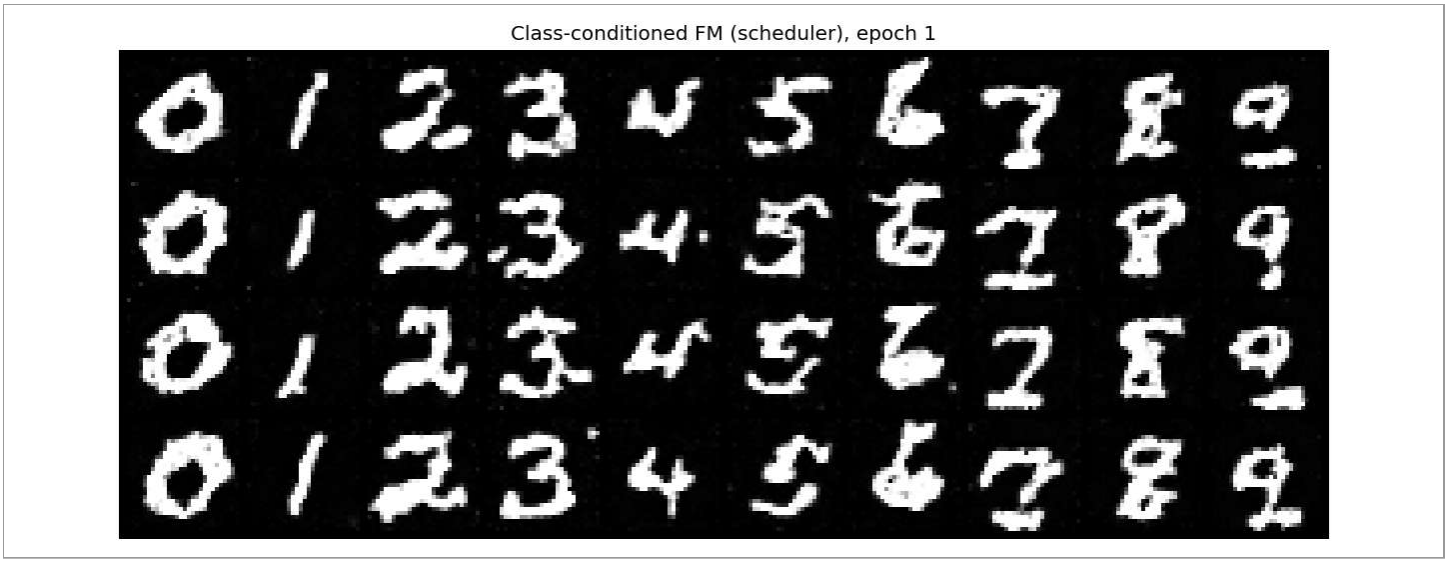


Fig B.2.6c | scheduler | epoch 1

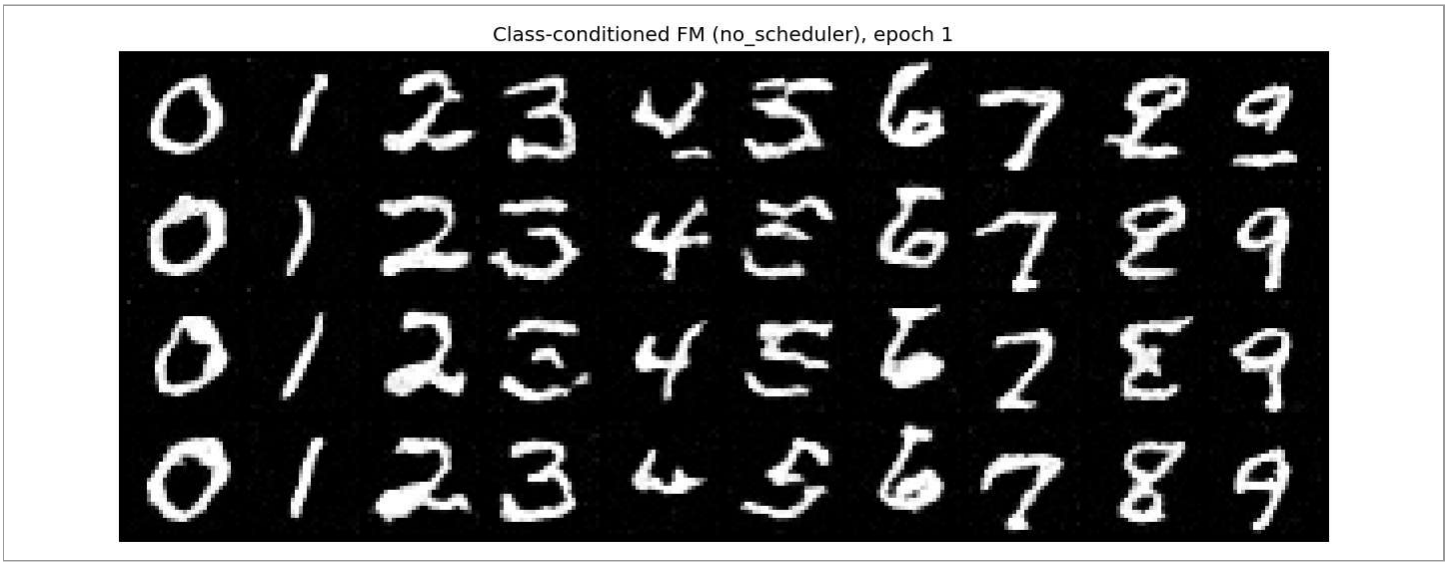


Fig B.2.6d | no scheduler | epoch 1

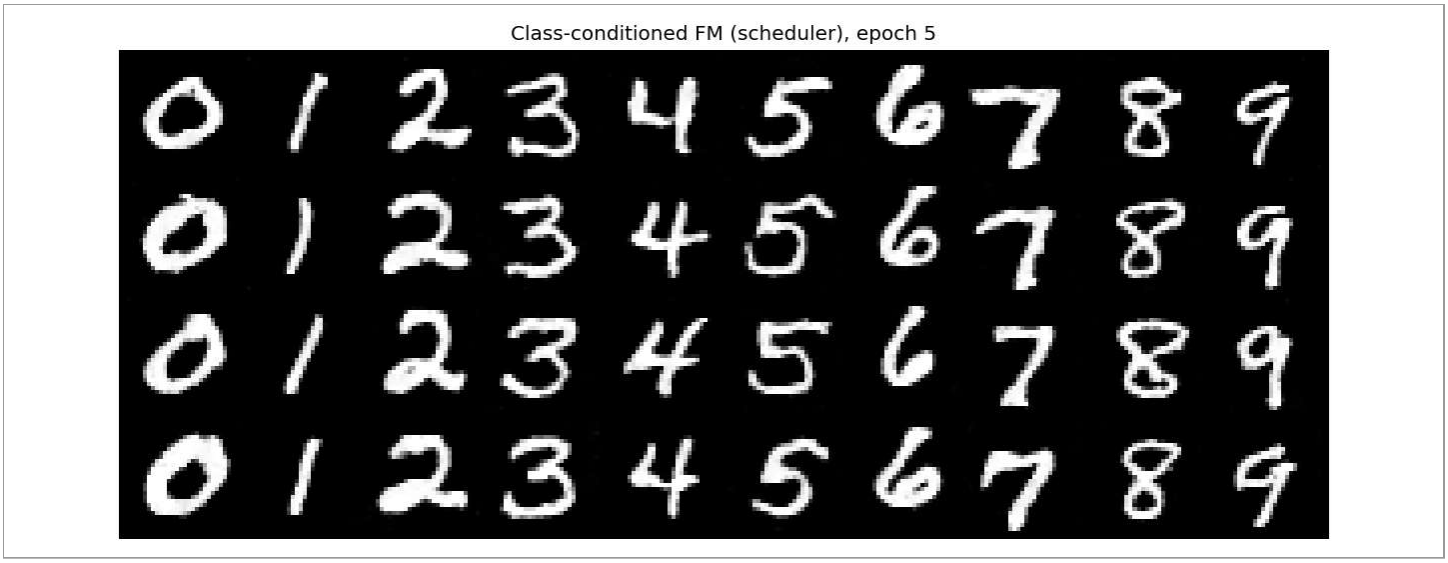


Fig B.2.6e | scheduler | epoch 5

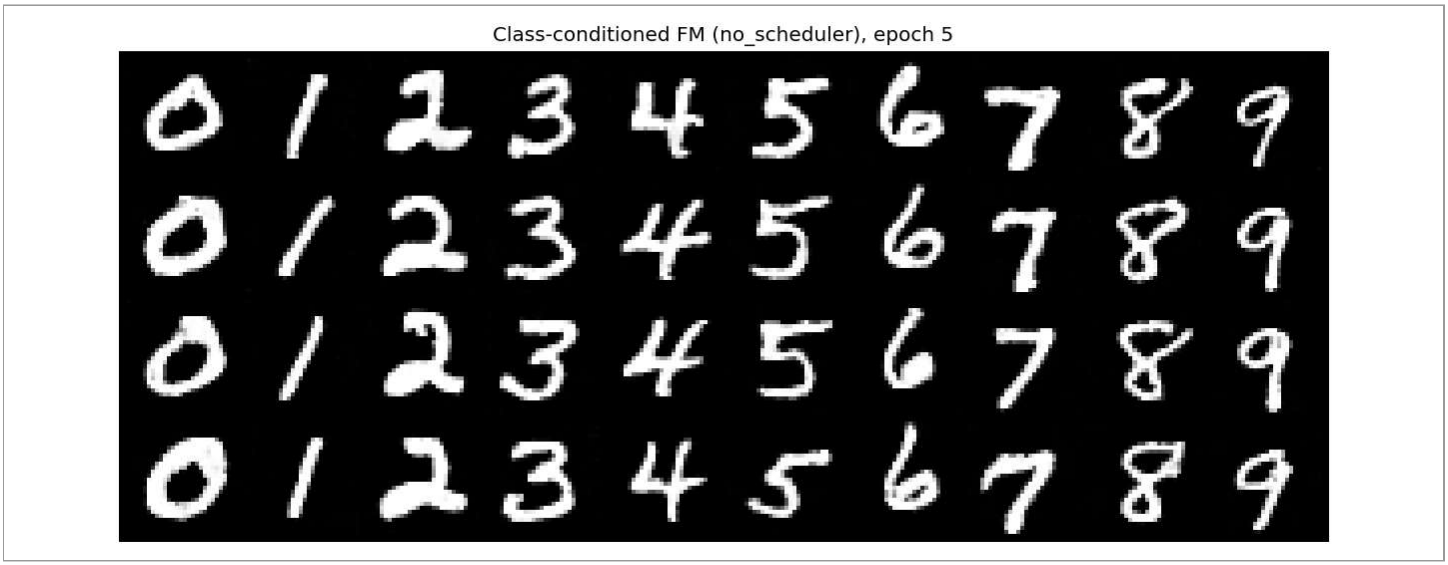


Fig B.2.6f | no scheduler | epoch 5

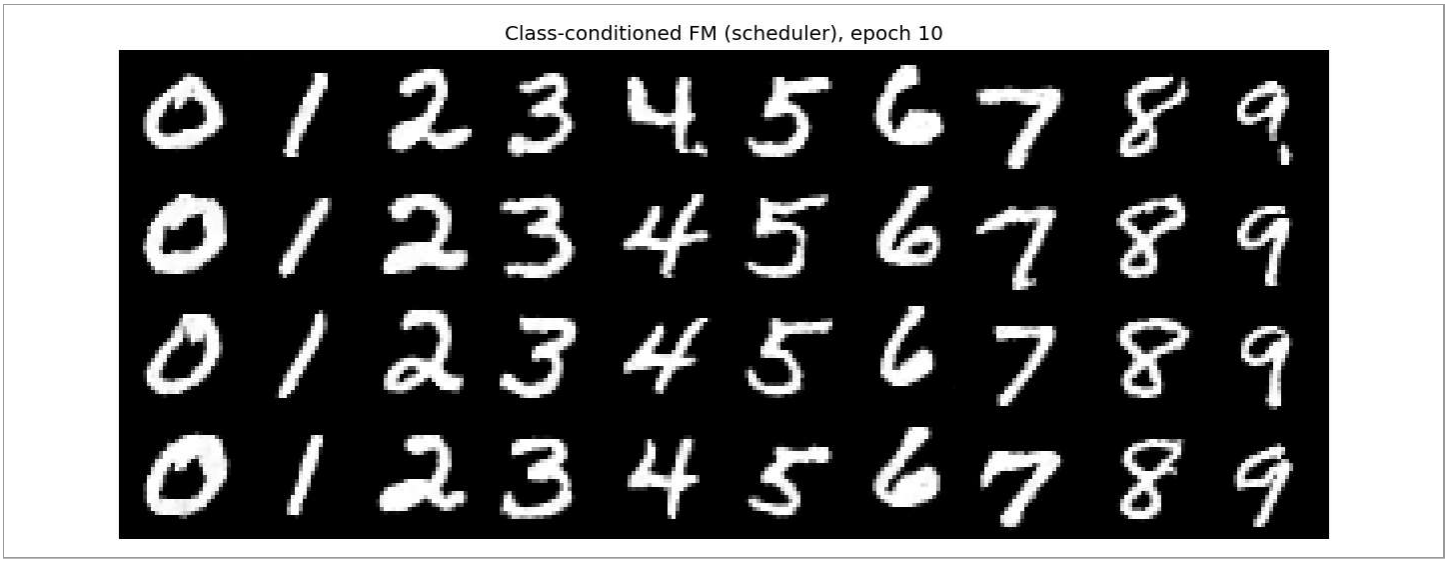


Fig B.2.6g | scheduler | epoch 10

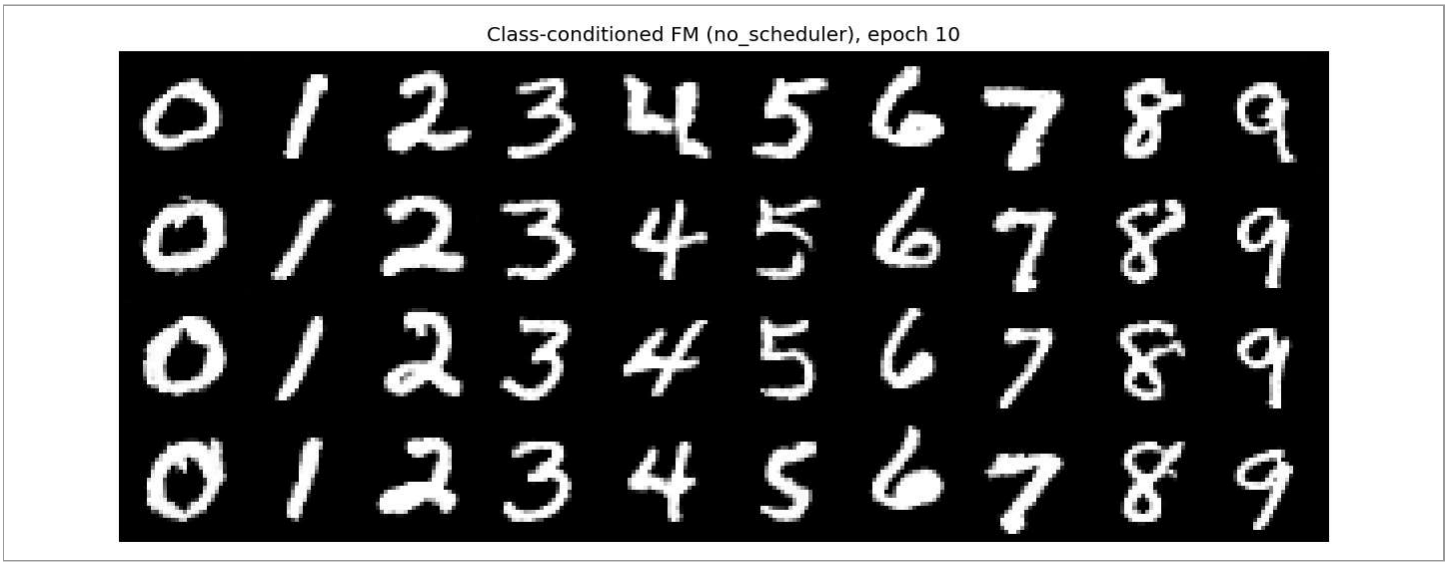


Fig B.2.6h | no scheduler | epoch 10

Learning rate choice: constant 2e-3

The scheduler decays the rate from 1e-2 to 1e-3 over 10 epochs: big steps early to learn rough shapes, small steps late to refine details. My constant 2e-3 sits between the start and end. It is small enough to refine details late without a decay schedule while still learning reasonably fast early. A constant 1e-2 would risk a noisy loss that doesn't settle, since big steps overshoot late in training. 2e-3 leans toward stability over early speed.

Comparison

- **Loss:** the scheduler run spikes to about 11.5 in the first steps (1e-2 is unstable early); the no-scheduler run starts near 1.2. Both settle around 0.08 to 0.1. The loss never reaches zero because at high noise many digits fit the same noisy image, a milder version of the pure-noise averaging.
- **Epoch 1:** scheduler samples are blobby, thick, and speckled with shaky 7s and 9s; no-scheduler samples are already cleaner and thinner.
- **Epoch 5:** both clean, every column the correct digit.
- **Epoch 10:** both clean; the scheduler run has small stray dots (the 4 and 9 in row 1), the no-scheduler run has one odd 4.

The high starting rate makes early training unstable, and the scheduler then shrinks it so the model settles. The constant 2e-3 avoids the early instability and reaches the same quality in 10 epochs. Class conditioning works: every column is the requested digit, in 4 different styles.

End

```
t = 1.00
```

Signal.

Classical blur can only trade noise for blur, a one-step model recovers layout and color but averages away detail, and iterative sampling recovers sharp, plausible detail through many small steps. Across both parts, MSE kept rewarding blurry averages, from the one-step estimate in Part A to the centroid digit in Part B, so looking at the results matters as much as the metric. Guidance and conditioning buy quality and control at the cost of variety.